claude-windows / dbc4cdae-04f6-4d06-af90-397bbce3fc57

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\subagents\agent-a44196c86bdb821e9.jsonl`
- SHA-256: `f5c7a6b12927316c07467dc55b28a93865dab48316cad9e9a500d3ae16b67819`
- Source modified: `2026-07-06T20:55:30+00:00`
- Imported at: `2026-07-08T15:59:36+00:00`
- project: `subagents`
- session_id: `dbc4cdae-04f6-4d06-af90-397bbce3fc57`

Transcript

1. user (2026-07-06T20:52:50.882Z)

Write a self-contained Python script `score_local.py` that scores WASB trajectory CSVs against golden rally labels using THIS repo's own windowing + metrics, so I can compare a torch-2.x GPU-resident-decode trajectory vs the CPU trajectory vs the cached baseline. It runs locally (Windows, Python 3.13) from the repo root `C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer` (imports already work: `from backend.pipeline.detectors.tracknet_runner import TrackNetRunner`, `from backend.eval import tolerance_metrics`, `from backend.eval.gt_loader import load_gt_intervals` all succeed). Output ONLY the final script in a ```python block + a short notes section. Do not run anything.

Read these to get the EXACT interfaces (cite what you use)

- `backend/pipeline/detectors/tracknet_runner.py` ? `TrackNetRunner.parse_trajectory_csv(path)` (returns points) and `TrackNetRunner.trajectory_to_action_windows(...)` ? get its FULL signature: what it needs (points, fps, frame_width, and any config object / velocity_thresh / merge_gap / min_window_duration), and what it returns (list of (start,end) seconds? objects with start/end?).
- `backend/eval/calibrate_wasb.py` ? it already wires parse_trajectory_csv -> trajectory_to_action_windows; copy its exact call pattern + the BASELINE config it uses `(0.02, 2.0, 2.0)`.
- `backend/eval/tolerance_metrics.py` ? the tolF1 scorer: the function that scores predicted intervals vs GT intervals (e.g. `score_intervals` / a tolF1 fn), its signature, and TAU_START/TAU_END (2.0/1.5). Use it for tol_f1/tol_precision/tol_recall.
- `backend/eval/rally_seg_eval.py` ? how it computes the tIoU-gated `f1@0.5` (and f1@0.25) on intervals vs GT (the greedy IoU-match). Reuse its scoring fn if importable, else replicate the simple greedy tIoU-F1 (match if temporal-IoU >= 0.5, 1:1).
- `backend/eval/gt_loader.py` ? `load_gt_intervals(path, strict=False)` for loading the `.rallies.csv`.
- `eval_baselines/nightly_baseline.json` ? find the `default`/`milestone` preset's per-video number for **`mahadevpura_1`** (`configs.default.per_video.mahadevpura_1.{f1@0.5, tol_f1}`) ? print it as a sanity anchor.

What score_local.py must do

- Hardcode-with-argv-override these inputs (all in the repo root or a `--dir`): trajectories `cpu_traj.csv`, `gpu_traj.csv`, `baseline_traj.csv`; labels `g1_labels.csv`; `--fps 59.94 --frame-width 1920 --name mahadevpura_1`.
- For EACH of the three trajectory CSVs: `parse_trajectory_csv` -> `trajectory_to_action_windows(...)` (using the repo's default/BASELINE windowing config exactly as calibrate_wasb does) -> predicted rally intervals. Load GT via `load_gt_intervals(g1_labels.csv)`.
- Score each predicted-vs-GT with BOTH: **tIoU f1@0.5** (and f1@0.25) and **tol_f1** (+ tol_precision/recall) using the repo's tolerance_metrics. Also print the predicted rally COUNT vs GT count.
- Print a clean table: rows = {baseline, cpu, gpu}, cols = {n_pred, f1@0.25, f1@0.5, tol_f1}. Print the eval_baselines anchor for mahadevpura_1 next to it. The KEY output is whether **gpu ? cpu ? baseline** (no regression).

- Robust: if an import/signature differs from my description, adapt to the ACTUAL code you read (the repo is source of truth). Wrap each trajectory in try/except so one failure doesn't kill the table.

Return the complete `score_local.py` + a notes section stating the exact functions/signatures you used and the windowing config, and the eval_baselines anchor value you found for mahadevpura_1.

2. assistant (2026-07-06T20:52:52.783Z)

I'll research the exact interfaces in this repo before writing the script.

3. user (2026-07-06T20:52:54.666Z)

```
1       """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3       This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4       inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6           1. Translate the Windows video path to its WSL-visible form (and optionally
7              stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8           2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9           3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10          4. Convert that trajectory into high-motion "action windows" ? the same
11             candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12             pipeline (AI handoff, DB population) is unchanged.
13
14      Nothing here imports TensorFlow; all model code stays in WSL.
15      """
16
17      import csv
18      import logging
19      import os
20      import shlex
21      import subprocess
22      from dataclasses import dataclass
23      from typing import Any, Dict, List, Optional, Tuple
24
25      from backend.config.models import IndexingConfig
26      from backend.pipeline.detectors.base import (
27          DetectorRunner,
28          WslCommandMixin,
29          wsl_tilde_quote,
30      )
31
32      logger = logging.getLogger(__name__)
33
34
35      @dataclass
36      class TrackNetConfig:
37          """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
38
39          distro: str = "Ubuntu"
40          repo_dir: str = "~/models/TrackNetV4"  # WSL path to the cloned repo
41          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
42          conda_env: str = "TrackNetV4"
43          weights_path: str = ""  # WSL path to the .h5/.keras weights (REQUIRED)
44          queue_length: int = 5
45          stage_in_wsl: bool = True  # copy video into WSL fs first (perf)
46          wsl_stage_dir: str = "~/clips"  # where staged videos/outputs live in WSL
47          timeout_sec: int = 1800  # 30 min; kills a hung call (0 = no timeout)
48          keep_frames: bool = False  # retain staged video after success (debug only)
49          min_free_gb: float = 0.0  # absolute floor (GB) atop the ~15×-source BLOCK guard (0
= off)
```

```
50
51          @classmethod
52          def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "TrackNetConfig":
53              if isinstance(idx_cfg, dict):
54                  idx_cfg = IndexingConfig(**idx_cfg)
55              tn = idx_cfg.tracknet
56              return cls(
57                  distro=tn.wsl_distro,
58                  repo_dir=tn.repo_dir,
59                  conda_sh=tn.conda_sh,
60                  conda_env=tn.conda_env,
61                  weights_path=tn.weights_path,
62                  queue_length=tn.queue_length,
63                  stage_in_wsl=tn.stage_in_wsl,
64                  wsl_stage_dir=tn.wsl_stage_dir,
65                  timeout_sec=tn.timeout_sec,
66                  keep_frames=tn.keep_frames,
67                  min_free_gb=tn.min_free_gb,
68              )
69
70
71      def to_wsl_mnt_path(win_path: str) -> str:
72          """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
73
74          Already-POSIX paths (starting with / or ~) are returned unchanged so the
75          same helper is safe to call on either kind of path.
76          """
77          p = str(win_path)
78          if p.startswith("/") or p.startswith("~"):
79              return p
80          p = p.replace("\\", "/")
81          if len(p) >= 2 and p[1] == ":":
82              drive = p[0].lower()
83              return f"/mnt/{drive}{p[2:]}"
84          return p
85
86
87      @dataclass
88      class TrajectoryPoint:
89          frame: int
90          visible: bool
91          x: float
92          y: float
93
94
95      class TrackNetRunner(WslCommandMixin, DetectorRunner):
96          """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
97
98          Implements the common DetectorRunner contract so a future Linux-native or
99          alternative trajectory runner can be substituted without touching the hybrids.
100         """
101
102         def __init__(self, cfg: TrackNetConfig):
103             self.cfg = cfg
104
105         def healthcheck(self) -> Tuple[bool, str]:
106             """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
107
108             Returns (ok, message). Cheap to call before a long run.
109             """
110             cmd = (
111                 f"conda activate {self.cfg.conda_env} && "
112                 f'python -c "import tensorflow as tf; '
113                 f"print('TF', tf.__version__, 'GPUs',
len(tf.config.list_physical_devices('GPU')))\""
```

```
            )
            try:
                res = self._wsl_bash(cmd)
            except FileNotFoundError:
                return (
                    False,
                    "`wsl` not found ? is this running on Windows with WSL2 installed?",
                )
            except subprocess.TimeoutExpired:
                return False, "WSL healthcheck timed out."
            out = (res.stdout or "").strip()
            if res.returncode != 0:
                return False, f"WSL env check failed: {(res.stderr or out).strip()}"
            return True, out

        # ------------------------------------------------------------------ inference

        def run_predict(
            self, video_win_path: str, output_win_dir: str, video_id: Optional[str] = None
        ) -> Optional[str]:
            """Run predict.py on a video. Returns the Windows path to the produced CSV, or
None.

            ``output_win_dir`` is a Windows directory (under the repo's output/) that
            WSL writes into via its /mnt view, so the Windows process can read the CSV
            back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
            therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
            a filename cannot collide on the output CSV.
            """
            if not self.cfg.weights_path:
                logger.error(
                    "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
                    "in config.json to the WSL path of your trained TrackNetV4 weights."
                )
                return None

            # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
            # relative paths through unchanged, so WSL would resolve them against the
            # predict.py cwd instead of the repo (same bug class as wasb_runner).
            output_win_dir = os.path.abspath(output_win_dir)
            os.makedirs(output_win_dir, exist_ok=True)
            # Normalize separators so basename/splitext work when tests (or collector)
            # pass Windows-style paths on a Linux host.
            video_win_path = str(video_win_path).replace("\\", "/")
            base = os.path.basename(video_win_path)
            stem, ext = os.path.splitext(base)

            # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
            if ext.lower() not in (".mp4", ".avi"):
                logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
                return None

            wsl_out = to_wsl_mnt_path(output_win_dir)
            # Namespace by video_id so two same-filename videos don't collide on the CSV.
            # predict.py derives its output name from the (staged) video stem, so staging
            # under the namespaced name propagates into "<key>_predict.csv".
            key = f"{stem}__{video_id[:12]}" if video_id else stem
            expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")

            # Resolve the video path WSL will read.
            if self.cfg.stage_in_wsl:
                staged = self._stage_video(video_win_path, f"{key}{ext}")
                if staged is None:
                    return None
                wsl_video = staged
```

```
178             else:
179                 # No staging: predict.py reads /mnt directly and writes
"<stem>_predict.csv".
180                 # We cannot rename its output, so fall back to the un-namespaced name.
181                 wsl_video = to_wsl_mnt_path(video_win_path)
182                 expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
183 
184             # Disk guard (D11): BLOCK before extraction if the WSL stage fs can't hold ~15×
the source
185             # (frame PNGs balloon the footprint). Best-effort measuring ? an undeterminable
read never
186             # blocks; min_free_gb (config) still acts as an absolute floor. See
base._wsl_frame_stage_shortfall.
187             shortfall = self._wsl_frame_stage_shortfall(video_win_path)
188             if shortfall:
189                 logger.error("TrackNet inference aborted ? %s", shortfall)
190                 return None
191 
192             cmd = (
193                 f"conda activate {self.cfg.conda_env} && "
194                 f"cd {self.cfg.repo_dir}/src && "
195                 f"python predict.py "
196                 f"--video_path {wsl_tilde_quote(wsl_video)} "
197                 f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
198                 f"--output_dir {wsl_tilde_quote(wsl_out)} "
199                 f"--queue_length {self.cfg.queue_length}"
200             )
201             logger.info("Running TrackNet inference on %s ...", base)
202             try:
203                 res = self._wsl_bash(cmd)
204             except subprocess.TimeoutExpired:
205                 logger.error(
206                     "TrackNet inference timed out after %ss.", self.cfg.timeout_sec
207                 )
208                 return None
209 
210             if res.returncode != 0:
211                 logger.error(
212                     "TrackNet predict.py failed:\n%s", (res.stderr or res.stdout)[-2000:]
213                 )
214                 return None
215 
216             if not os.path.exists(expected_csv_win):
217                 logger.error(
218                     "TrackNet finished but expected CSV not found at %s", expected_csv_win
219                 )
220                 return None
221             logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
222 
223             # Success ? the CSV is the durable output; drop the staged video copy (a pure,
224             # regenerable intermediate). On earlier failure we return above without
cleaning.
225             if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
226                 logger.info(
227                     "Cache hygiene: removing staged video for %s "
228                     "(set indexing.tracknet.keep_frames=true to retain).",
229                     key,
230                 )
231                 self._wsl_rm_rf([wsl_video])
232             return expected_csv_win
233 
234         def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
235             """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
236 
237             Returns the WSL path to the staged copy, or None on failure.
```

```
            """
            src_mnt = to_wsl_mnt_path(video_win_path)
            reason = self.wsl_source_unreadable_reason(src_mnt)
            if reason:
                logger.error("Cannot stage video into WSL: %s", reason)
                return None
            dest = f"{self.cfg.wsl_stage_dir}/{base}"
            cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
            try:
                res = self._wsl_bash(cmd)
            except subprocess.TimeoutExpired:
                logger.error("Staging video into WSL timed out.")
                return None
            if res.returncode != 0 or "OK" not in (res.stdout or ""):
                logger.error(
                    "Failed to stage video into WSL: %s", (res.stderr or res.stdout).strip()
                )
                return None
            return dest

        # ------------------------------------------------------------ CSV -> windows

        @staticmethod
        def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
            """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
            points: List[TrajectoryPoint] = []
            with open(csv_win_path, newline="") as f:
                reader = csv.DictReader(f)
                for row in reader:
                    try:
                        points.append(
                            TrajectoryPoint(
                                frame=int(row["Frame"]),
                                visible=int(row["Visibility"]) == 1,
                                x=float(row["X"]),
                                y=float(row["Y"]),
                            )
                        )
                    except (KeyError, ValueError):
                        continue
            points.sort(key=lambda p: p.frame)
            return points

        @staticmethod
        def _active_frames(
            points: List[TrajectoryPoint],
            fps: float,
            frame_width: float,
            velocity_thresh: float,
        ) -> List[Tuple[int, float]]:
            """Per-frame ``(frame_idx, velocity)`` for frames where the shuttle is in
flight.

            Computes each visible point's normalised velocity from the last *visible* point;
            an absent shuttle breaks the trajectory. A frame qualifies when its velocity
exceeds
            ``velocity_thresh``. ``fps``/``frame_width`` are expected to be already
defaulted by
            the caller so identical floats feed the velocity math."""
            active = []  # (frame_idx, velocity)
            prev: Optional[TrajectoryPoint] = None
            for p in points:
                if not p.visible:
                    prev = None  # break the trajectory; absent shuttle = not in flight
```

```
                    continue
                if prev is not None:
                    dframes = p.frame - prev.frame
                    if dframes > 0:
                        dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
                        velocity = (dist / frame_width) * (fps / dframes)
                        if velocity > velocity_thresh:
                            active.append((p.frame, velocity))
                prev = p
            return active

        @staticmethod
        def trajectory_to_action_windows(
            points: List[TrajectoryPoint],
            fps: float,
            frame_width: float,
            chunk_start: float = 0.0,
            velocity_thresh: float = 0.02,
            merge_gap: float = 2.0,
            min_window_duration: float = 2.0,
            chunk_index: Optional[int] = None,
            max_window_duration: float = 0.0,
            min_split_gap: float = 0.5,
            window_hard_cap: bool = False,
            window_inactivity_end: float = 0.0,
            inactivity_min_density: float = 0.34,
            inactivity_rally_thresh: float = 0.08,
            inactivity_temporal_density: bool = False,
            window_blob_fallback: bool = False,
            window_blob_factor: float = 4.0,
        ) -> List[Dict[str, Any]]:
            """Convert a shuttle trajectory into candidate rally windows.

            A frame is "active" when the shuttle is visible AND its inter-frame
            displacement (normalised by frame width, per second) exceeds
            ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
            ``merge_gap`` seconds of each other are grouped into one window; short
            windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
            shape feeding the AI-handoff phase is identical.

            ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
against
            the *over-merge* failure where "shuttle moving" is conflated with "rally in
progress"
            and a single window swallows several rallies + the dead time between them. When
set, any
            window longer than this is recursively SPLIT at its largest internal inactivity
gap (the
            longest stretch with no in-flight shuttle ? the most likely rally boundary),
provided
            that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
fix from
            ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
recall cannot
            drop, and it is config-gated default-OFF until measured on the golden set.

            ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
an
            over-long window has NO internal inactivity gap to split on (a continuously-
active
            trajectory ? the shuttle never stops moving, e.g. the real-footage
``mahadevpura-1``
            blob where the gap-splitter alone left a single 174 s window), it is recursively
            hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
cuts
```

```
353             (never merges), so recall cannot drop; gated default-OFF until measured.
354
355             ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
unlike
356             ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
every gap-less
357             over-cap window and over-segments normal clips), this runs AFTER the R4
inactivity trim and
358             force-chops ONLY a genuine *blob* ? a group still longer than
``window_blob_factor`` ×
359             ``max_window_duration`` once the gap-split and the principled rally-end trim
have both had
360             their chance (e.g. mahadevpura-1's ~65 s residual ? 11× a 6 s cap; a 7 s rally
is left
361             alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
logged, and
362             each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
that needs
363             rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
(recall
364             can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
measured.
365             """
366             if fps <= 0:
367                 fps = 30.0
368             if frame_width <= 0:
369                 frame_width = 1280.0
370
371             active = TrackNetRunner._active_frames(
372                 points, fps, frame_width, velocity_thresh
373             )
374
375             if not active:
376                 return []
377
378             max_gap_frames = int(merge_gap * fps)
379
380             def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
381                 vels = [v for _, v in group]
382                 return {
383                     "start": group[0][0] / fps + chunk_start,
384                     "end": group[-1][0] / fps + chunk_start,
385                     "active_frame_count": len(group),
386                     "peak_velocity": max(vels),
387                     "mean_velocity": sum(vels) / len(vels),
388                     "track_id_count": 1,  # single shuttle; kept for window-shape parity
389                     "chunk_index": chunk_index,
390                 }
391
392             groups: List[List[Tuple[int, float]]] = []
393             group = [active[0]]
394             for af in active[1:]:
395                 if af[0] - group[-1][0] <= max_gap_frames:
396                     group.append(af)
397                 else:
398                     groups.append(group)
399                     group = [af]
400             groups.append(group)
401
402             if max_window_duration > 0:
403                 groups = TrackNetRunner._split_overlong_groups(
404                     groups,
405                     int(max_window_duration * fps),
406                     int(min_split_gap * fps),
407                     hard_cap=window_hard_cap,
```

```
                )

            # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
            # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
            # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
            # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
            # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
            if window_inactivity_end > 0:
                tw = int(window_inactivity_end * fps)
                groups = [
                    TrackNetRunner._trim_inactive_tail(
                        g,
                        tw,
                        inactivity_min_density,
                        inactivity_rally_thresh,
                        temporal_density=inactivity_temporal_density,
                    )
                    for g in groups
                ]

            # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
            # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
            # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
            # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
            forced_flags = [False] * len(groups)
            if window_blob_fallback and max_window_duration > 0:
                groups, forced_flags = TrackNetRunner._blob_fallback_chop(
                    groups, int(max_window_duration * fps), window_blob_factor
                )

            windows = []
            for g, forced in zip(groups, forced_flags):
                w = _finalize(g)
                if forced:
                    w["forced_cut"] = True
                windows.append(w)
            return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

        @staticmethod
        def _trim_inactive_tail(
            group: List[Tuple[int, float]],
            trail_frames: int,
            min_density: float,
            rally_thresh: float,
            temporal_density: bool = False,
        ) -> List[Tuple[int, float]]:
            """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
*slowly*
            (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
windowing
            over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
``rally_thresh``
            (higher than the active threshold). The rally is "on" at frame ``i`` while its
trailing
            ``trail_frames`` window holds >= ``min_density`` fraction of fast frames; trim
back to the
```

```
            LAST such frame ? everything after is sustained slow drift (rally over). Only
cuts (recall
            can't rise); a window whose tail never goes slow is untouched, and if no fast-
dense point
            exists at all the group is kept whole (fail-safe, no over-trim). O(n) two-
pointer; the START
            is left alone (already ~Gemini-accurate per the n=2 boundary-error finding).

            ``temporal_density`` (default False = legacy/promoted behaviour) selects the
density
            DENOMINATOR (code-review finding B). ``group`` carries only active frames, so
the legacy
            denominator ? the COUNT of active samples in the trailing window (``i - lo +
1``) ? reads a
            lone fast sample after a long tracking gap as 100% dense and holds the tail
open. When True,
            the denominator is the trailing window's TEMPORAL length instead, so sparse
tracking can't
            inflate density. With DENSE tracking the two are identical, so this is a no-op
on
            well-tracked clips ? hence it ships flag-gated default-OFF until measured on the
golden set
            at the SERVED operating point (R4 is currently default-ON in prod; see
RALLY_DETECTION_FIXES_PLAN)."""
            if trail_frames <= 0 or len(group) < 2:
                return group
            frames = [f for f, _ in group]
            fast = [1 if v > rally_thresh else 0 for _, v in group]
            lo = 0
            fast_sum = 0
            last_dense = -1
            for i in range(len(frames)):
                fast_sum += fast[i]
                while frames[lo] <= frames[i] - trail_frames:
                    fast_sum -= fast[lo]
                    lo += 1
                if temporal_density:
                    # Trailing window's TEMPORAL length: full ``trail_frames`` once that
much footage
                    # has elapsed, else the footage seen so far. Sparse tracking can't
inflate it.
                    denom = min(trail_frames, frames[i] - frames[0] + 1)
                else:
                    denom = (
                        i - lo + 1
                    )  # legacy: count of active samples in the trailing window
                if denom > 0 and fast_sum / denom >= min_density:
                    last_dense = i
            return group[: last_dense + 1] if last_dense >= 0 else group

        @staticmethod
        def _midpoint_split_index(group: List[Tuple[int, float]]) -> int:
            """Index of the active frame nearest the group's temporal midpoint (used to chop
a
            gap-less group into two). Shared by the W1 hard-cap fallback and the R5 blob-
fallback."""
            mid = (group[0][0] + group[-1][0]) / 2.0
            split_i = min(range(1, len(group)), key=lambda i: abs(group[i][0] - mid))
            return split_i

        @staticmethod
        def _split_overlong_groups(
            groups: List[List[Tuple[int, float]]],
            max_win_frames: int,
            min_split_frames: int,
```

```
            hard_cap: bool = False,
        ) -> List[List[Tuple[int, float]]]:
            """Recursively split any active-frame group longer than ``max_win_frames`` at
its largest
            internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
likely rally
            boundary. Splitting only subdivides existing groups (never merges/extends), so
it cannot
            invent or drop coverage; a group with no gap ? the floor is left intact (a
genuinely long
            rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at
its
            temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
true upper
            bound on continuously-active trajectories). Iterative worklist to bound stack
depth."""
            if max_win_frames <= 0:
                return groups
            out: List[List[Tuple[int, float]]] = []
            work = list(groups)
            while work:
                g = work.pop()
                if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
                    out.append(g)
                    continue
                best_i, best_gap = -1, -1
                for i in range(1, len(g)):
                    gap = g[i][0] - g[i - 1][0]
                    if gap > best_gap:
                        best_gap, best_i = gap, i
                if best_i < 0 or best_gap < min_split_frames:
                    # Too long but no real dead-time gap. Default: keep as one (a genuine
long rally).
                    # hard_cap: chop at the temporal midpoint so the cap is actually
enforced.
                    if hard_cap:
                        split_i = TrackNetRunner._midpoint_split_index(g)
                        work.append(g[:split_i])
                        work.append(g[split_i:])
                    else:
                        out.append(g)
                    continue
                work.append(g[:best_i])
                work.append(g[best_i:])
            out.sort(key=lambda grp: grp[0][0])
            return out

        @staticmethod
        def _blob_fallback_chop(
            groups: List[List[Tuple[int, float]]],
            max_win_frames: int,
            blob_factor: float = 4.0,
        ) -> Tuple[List[List[Tuple[int, float]]], List[bool]]:
            """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split
AND the R4
            inactivity trim have each had their chance, a group still longer than
``blob_factor`` ×
            ``max_win_frames`` is a genuine continuously-active blob (no internal gap, no
rally-end
            signal the principled rules could find ? the real-footage ``mahadevpura-1`` ~65
s residual ?
            11× a 6 s cap). The factor is what keeps this SURGICAL: a normal long rally
(~1?2× the cap)
            is left ALONE; only the degenerate blob qualifies. As a LAST RESORT the blob is
            midpoint-chopped to ? the cap and every resulting piece is FLAGGED as a forced
```

```
cut, so the
560             degenerate single-window outcome is avoided and the clip is surfaced (logged) as
needing
561             rally-STATE cues ? net-crossings / two-sided motion ? not a bigger cap. Only
cuts (recall
562             can't drop). Returns ``(groups, forced_flags)`` aligned 1:1; ``forced_flags[i]``
is True iff
563             that piece came from a forced chop. Iterative worklist to bound stack depth."""
564             if max_win_frames <= 0:
565                 return groups, [False] * len(groups)
566             blob_frames = max_win_frames * max(1.0, blob_factor)
567             out: List[List[Tuple[int, float]]] = []
568             flags: List[bool] = []
569             forced = 0
570             for g in groups:
571                 if len(g) < 2 or (g[-1][0] - g[0][0]) <= blob_frames:
572                     out.append(g)
573                     flags.append(False)
574                     continue
575                 pieces: List[List[Tuple[int, float]]] = []
576                 work = [g]
577                 while work:
578                     h = work.pop()
579                     if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
580                         pieces.append(h)
581                         continue
582                     split_i = TrackNetRunner._midpoint_split_index(h)
583                     work.append(h[:split_i])
584                     work.append(h[split_i:])
585                 pieces.sort(key=lambda grp: grp[0][0])
586                 out.extend(pieces)
587                 flags.extend([True] * len(pieces))
588                 forced += len(pieces) - 1
589             if forced:
590                 logger.warning(
591                     "R5 blob-fallback: forced %d midpoint cut(s) on continuously-active
group(s) with "
592                     "no rally-end signal ? this clip needs rally-STATE cues (net-crossings /
two-sided "
593                     "motion), not a bigger cap",
594                     forced,
595                 )
596             return out, flags
597
```

4. user (2026-07-06T20:52:55.293Z)

```
1       """Per-video calibration driver: WASB trajectory + golden rallies -> tuned thresholds +
"+X%".
2
3       Wires the already-tested pieces together:
4         WASB trajectory CSV (Frame,Visibility,X,Y)
5           -> TrackNetRunner.trajectory_to_action_windows(cfg)   [predict_fn]
6           -> backend.eval.calibration (improvement_report / cross_validate)
7
8       Run (after a full-video WASB pass exists):
9           python -m backend.eval.calibrate_wasb --trajectory full_traj.csv \
10              --labels "golden_labels_badminton.csv" --fps 59.94 --frame-width 1920
11
12      Honesty note: the grid is *selected* on the full GT, so `improvement_report` on that
13      same GT is an **optimistic** upper bound. The trustworthy number is the
14      cross-validated held-out **recall** (and, once a 2nd labelled video exists,
15      `leave_one_video_out` for precision/F1 generalization).
16      """
17
```

```
    from __future__ import annotations

    import argparse
    from typing import Callable, List, Tuple

    from backend.eval.calibration import (
        cross_validate,
        evaluate,
        improvement_report,
        select_best,
    )
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

    Interval = Tuple[float, float]
    Config = Tuple[float, float, float]  # (velocity_thresh, merge_gap, min_window_duration)

    BASELINE: Config = (
        0.02,
        2.0,
        2.0,
    )  # the WasbConfig / trajectory_to_action_windows defaults


    def load_golden_gts(csv_path: str) -> List[Interval]:
        """Rally [start,end] seconds from a golden CSV ? delegates to the canonical loader.

        Accepts BOTH golden conventions now (start_time/end_time AND the collector export's
        start,end ? the historical fork is closed; see gt_loader.py / ADR-008). Keeps this
        driver's legacy lenient semantics (skip bad rows), but skipped rows are now logged.
        """
        from backend.eval.gt_loader import load_gt_intervals

        return load_gt_intervals(csv_path, strict=False)


    def make_predict_fn(
        trajectory_csv: str, fps: float, frame_width: float
    ) -> Callable[[Config], List[Interval]]:
        """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)

        def predict_fn(cfg: Config) -> List[Interval]:
            vt, mg, mwd = cfg
            wins = TrackNetRunner.trajectory_to_action_windows(
                points,
                fps=fps,
                frame_width=frame_width,
                velocity_thresh=vt,
                merge_gap=mg,
                min_window_duration=mwd,
            )
            return [(w["start"], w["end"]) for w in wins]

        return predict_fn


    def default_grid() -> List[Config]:
        return [
            (vt, mg, mwd)
            for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
            for mg in (1.0, 2.0, 3.0)
            for mwd in (1.0, 2.0, 3.0)
        ]
```

```
    def run(
        trajectory_csv: str,
        labels_csv: str,
        fps: float,
        frame_width: float,
        iou: float = 0.5,
    ) -> dict:
        gts = load_golden_gts(labels_csv)
        if not gts:
            raise SystemExit(f"no usable golden rallies in {labels_csv}")
        predict_fn = make_predict_fn(trajectory_csv, fps, frame_width)
        grid = default_grid()

        base = evaluate(predict_fn(BASELINE), gts, iou)
        best_cfg, best_f1 = select_best(
            predict_fn, grid, gts, metric="f1", iou_threshold=iou
        )
        fit = improvement_report(
            predict_fn, BASELINE, best_cfg, gts, iou
        )  # OPTIMISTIC (fit on full GT)
        cv = cross_validate(predict_fn, grid, gts, k=5)  # HONEST held-out recall

        print("=== WASB rally-segmentation calibration ===")
        print(f"golden rallies: {len(gts)}  | IoU thresh: {iou}")
        print(
            f"baseline cfg {BASELINE}: precision={base.precision:.3f}
recall={base.recall:.3f} "
            f"f1={base.f1:.3f} meanIoU={base.mean_iou:.3f}"
        )
        print(f"tuned cfg    {best_cfg}: f1={best_f1:.3f}")
        print(
            f"  precision {fit['baseline']['precision']:.3f} ->
{fit['tuned']['precision']:.3f} "
            f"({fit['precision_delta_pct']:+.1f}% , fit-on-full = optimistic)"
        )
        print(
            f"  f1        {fit['baseline']['f1']:.3f} -> {fit['tuned']['f1']:.3f} "
            f"({fit['f1_delta_pct']:+.1f}%)"
        )
        print(
            f"HONEST (cross-val, {cv.get('folds', '?')} folds): held-out recall "
            f"{cv.get('mean_test', float('nan')):.3f} ± {cv.get('std_test', 0):.3f}, "
            f"generalization_gap {cv.get('generalization_gap', float('nan')):+.3f}"
        )
        print(
            "(For precision/F1 generalization, add a 2nd labelled video ->
leave_one_video_out.)"
        )
        return {"baseline": base.as_dict(), "best_cfg": best_cfg, "fit": fit, "cv": cv}


    def main():
        ap = argparse.ArgumentParser()
        ap.add_argument(
            "--trajectory", required=True, help="WASB trajectory CSV (Frame,Visibility,X,Y)"
        )
        ap.add_argument(
            "--labels", required=True, help="golden labels CSV (start_time/end_time cols)"
        )
        ap.add_argument("--fps", type=float, default=59.94)
        ap.add_argument("--frame-width", type=float, default=1920.0)
        ap.add_argument("--iou", type=float, default=0.5)
        args = ap.parse_args()
        run(args.trajectory, args.labels, args.fps, args.frame_width, args.iou)

```

```
145
146     if __name__ == "__main__":
147         main()
148
```

5. user (2026-07-06T20:52:55.791Z)

```
1       """R2 ? task-faithful rally-boundary eval (tolerance-match + over-segmentation guard).
2
3       The headline rally metric (tIoU-F1 is kept as a SECONDARY diagnostic): a prediction
matches a
4       golden rally iff ``|?start| ? ?_start AND |?end| ? ?_end`` under a **strict 1:1
assignment**, so
5       over-production costs precision directly. Reported as a DECOMPOSITION ? ``P/R/F1@?`` +
start/end
6       boundary-error distributions + an over-seg guard (split/merge counts) + a ?-sweep ?
never one
7       number (a single ? misleads; see the design doc).
8
9       Locked decisions + rationale: ``docs/R2_EVAL_METRIC_DESIGN.md`` (owner-reviewed
2026-06-17):
10      asymmetric ? (``?_start=2.0`` forgives the ~1?1.5 s service window; ``?_end=1.5`` keeps
the
11      rally-end honest ? IAA-calibrated later); augment tIoU now, ablation-gate any eventual
swap.
12
13      Pure + stdlib; **numpy/scipy are OPTIONAL** ? used for the exact Hungarian assignment
when present,
14      else a deterministic greedy 1:1 fallback. Both agree on the temporally-ordered, near-
diagonal
15      eligibility graphs that rally intervals produce, so scores are environment-independent.
16      """
17
18      from __future__ import annotations
19
20      import statistics
21      from typing import Any, Dict, List, Optional, Tuple
22
23      Interval = Tuple[float, float]
24      #: (pred_idx, gt_idx, dstart_signed, dend_signed) ? signed = pred ? gt (positive = pred
is late).
25      Match = Tuple[int, int, float, float]
26
27      #: Provisional ? (the design's locked starting point; fixed by the IAA study later).
Always
28      #: report the sweep alongside, since the golden END convention carries >1.5 s of noise.
29      TAU_START: float = 2.0
30      TAU_END: float = 1.5
31      SWEEP_TAUS: Tuple[float, ...] = (1.5, 2.0, 2.5, 3.0)
32
33
34      def _eligible(p: Interval, g: Interval, tau_start: float, tau_end: float) -> bool:
35          return abs(p[0] - g[0]) <= tau_start and abs(p[1] - g[1]) <= tau_end
36
37
38      def _greedy_match_1to1(
39          preds: List[Interval], gts: List[Interval], tau_start: float, tau_end: float
40      ) -> List[Match]:
41          """Deterministic greedy 1:1: eligible (pred, gt) pairs in ascending combined
boundary error,
42          assigned first-come and skipping used indices. Optimal-or-near on the near-diagonal
eligibility
43          graph rally intervals produce. The numpy/scipy-free path (and the CI path)."""
44          pairs: List[Tuple[float, float, int, int]] = []
45          for i, p in enumerate(preds):
```

```
46              for j, g in enumerate(gts):
47                  if _eligible(p, g, tau_start, tau_end):
48                      # sort key: combined error, then (i, j) for a fully deterministic
tiebreak
49                      pairs.append(
50                          (
51                              abs(p[0] - g[0]) + abs(p[1] - g[1]),
52                              float(i * 1e-6 + j * 1e-9),
53                              i,
54                              j,
55                          )
56                      )
57          pairs.sort()
58          used_p: set = set()
59          used_g: set = set()
60          matches: List[Match] = []
61          for _, _, i, j in pairs:
62              if i in used_p or j in used_g:
63                  continue
64              used_p.add(i)
65              used_g.add(j)
66              matches.append((i, j, preds[i][0] - gts[j][0], preds[i][1] - gts[j][1]))
67          matches.sort(key=lambda mm: mm[1])
68          return matches
69
70
71      def _hungarian_match_1to1(
72          preds: List[Interval], gts: List[Interval], tau_start: float, tau_end: float
73      ) -> Optional[List[Match]]:
74          """Exact max-cardinality (then min total boundary error) via scipy. Returns ``None``
when
75          numpy/scipy are unavailable so the caller falls back to greedy."""
76          try:
77              import numpy as np
78              from scipy.optimize import linear_sum_assignment
79          except Exception:
80              return None
81          n, m = len(preds), len(gts)
82          size = max(n, m)
83          big = 1.0e6
84          cost = np.full((size, size), big, dtype=float)
85          for i, p in enumerate(preds):
86              for j, g in enumerate(gts):
87                  if _eligible(p, g, tau_start, tau_end):
88                      cost[i, j] = abs(p[0] - g[0]) + abs(p[1] - g[1])
89          rows, cols = linear_sum_assignment(cost)
90          matches: List[Match] = []
91          for i, j in zip(rows.tolist(), cols.tolist()):
92              if i < n and j < m and cost[i, j] < big:  # drop forced ineligible assignments
93                  matches.append((i, j, preds[i][0] - gts[j][0], preds[i][1] - gts[j][1]))
94          matches.sort(key=lambda mm: mm[1])
95          return matches
96
97
98      def match_1to1(
99          preds: List[Interval],
100         gts: List[Interval],
101         tau_start: float = TAU_START,
102         tau_end: float = TAU_END,
103     ) -> Tuple[List[Match], float, float, float]:
104         """Strict 1:1 tolerance match ? ``(matches, precision, recall, f1)``. ``P = matched
/ n_pred``
105         (over-production costs precision), ``R = matched / n_gt``. Exact Hungarian when
scipy is
106         present; deterministic greedy otherwise."""
```

```
        n, m = len(preds), len(gts)
        if n == 0 or m == 0:
            return [], 0.0, 0.0, 0.0
        matches = _hungarian_match_1to1(preds, gts, tau_start, tau_end)
        if matches is None:
            matches = _greedy_match_1to1(preds, gts, tau_start, tau_end)
        k = len(matches)
        precision = k / n
        recall = k / m
        f1 = (
            (2 * precision * recall / (precision + recall)) if (precision + recall) else 0.0
        )
        return matches, precision, recall, f1


    def _percentile(xs: List[float], p: float) -> float:
        """Linear-interpolated percentile (pure stdlib)."""
        if not xs:
            return 0.0
        s = sorted(xs)
        k = (len(s) - 1) * p / 100.0
        lo = int(k)
        hi = min(lo + 1, len(s) - 1)
        return s[lo] + (s[hi] - s[lo]) * (k - lo)


    def _overlap(a: Interval, b: Interval) -> float:
        return max(0.0, min(a[1], b[1]) - max(a[0], b[0]))


    def boundary_error_report(
        preds: List[Interval], gts: List[Interval]
    ) -> Dict[str, Dict[str, float]]:
        """Median + IQR of signed AND absolute ?start, ?end, split start/end ? the "start
solved,
        end open" diagnostic that aims R4.

        Computed over **best-overlap** matches (each GT ? its maximally-overlapping
prediction, any
        positive overlap), NOT the within-? 1:1 matches: a ?-gated set is survivorship-
biased (loose
        ends fail the gate and vanish from the distribution), which would HIDE the very end-
deficit
        this diagnostic exists to surface. Unconditional best-overlap is what the day-1
validation used
        to find |?end| ? |?start|. (The within-? deficit shows up instead as low
``tol_recall``.)"""

        def dist(vals: List[float]) -> Dict[str, float]:
            av = [abs(v) for v in vals]
            return {
                "signed_median": statistics.median(vals) if vals else 0.0,
                "abs_median": statistics.median(av) if av else 0.0,
                "abs_p25": _percentile(av, 25.0),
                "abs_p75": _percentile(av, 75.0),
                "n": float(len(vals)),
            }

        ds: List[float] = []
        de: List[float] = []
        for g in gts:
            best = max(preds, key=lambda p: _overlap(p, g), default=None)
            if best is not None and _overlap(best, g) > 0.0:
                ds.append(best[0] - g[0])
                de.append(best[1] - g[1])
```

```
166         return {"dstart": dist(ds), "dend": dist(de)}
167
168
169     def over_seg_report(preds: List[Interval], gts: List[Interval]) -> Dict[str, int]:
170         """The over-segmentation GUARD (per-video no-regression in promotion):
``split_count`` (?2
171         preds covering one rally) + ``merge_count`` (one pred ? ?2 rallies). Reuses the
bipartite
172         overlap-graph in ``segmentation_metrics.merge_split_report``."""
173         from backend.eval.segmentation_metrics import merge_split_report
174
175         r = merge_split_report(preds, gts)
176         return {"split_count": int(r["splits"]), "merge_count": int(r["merges"])}
177
178
179     # ------------------------------------------------------------------------ #
180     # R1 ? net over-segmentation promotion guard
181     # ------------------------------------------------------------------------ #
182     #: Default over-seg cost weights. A MERGE hides a rally (?2 golden rallies collapse into
one
183     #: predicted window ? the user can't reach the hidden one ? recall loss at rally
granularity); a
184     #: SPLIT only fragments one rally (it is still found, just in pieces). So a merge is the
strictly
185     #: worse fault and is weighted higher. (Equal weights also pass the milestone ? see the
R1 evidence;
186     #: the asymmetry is the principled default, not a number tuned to force a verdict.)
187     WEIGHT_MERGE: float = 2.0
188     WEIGHT_SPLIT: float = 1.0
189
190
191     def over_seg_guard(
192         base: List[Dict[str, Any]],
193         cand: List[Dict[str, Any]],
194         *,
195         weight_merge: float = WEIGHT_MERGE,
196         weight_split: float = WEIGHT_SPLIT,
197         use_rates: bool = True,
198         eps: float = 1e-9,
199     ) -> Dict[str, Any]:
200         """**Net** over-segmentation promotion guard (the R1 refinement of the strict per-
video rule).
201
202         ``base``/``cand`` are aligned per-video over-seg dicts (the rows ``rally_seg_eval``
already
203         emits): each needs ``split_count``/``merge_count`` and ? when ``use_rates``
(default) ?
204         ``merge_rate``/``split_rate``. Optional ``name`` keys align the two lists by video
(else by
205         position). Returns the promote-or-block verdict for promoting ``cand`` over
``base``.
206
207         **Why the strict rule needed refining.** R2 §2.3.3's guard blocks promotion if
``cand`` raises
208         ``split_count`` OR ``merge_count`` on ANY video. That is right for an *incremental*
cue (same
209         windowing structure, an increase = a real regression) but mis-fires on a
*structural* change
210         (mega-windows ? bounded): splitting a single mega-window necessarily lifts
``split_count`` from
211         ~0, and the raw merge **count** is non-monotonic ? one mega-window swallowing 40
rallies is
212         ``merge_count=1`` yet ``merge_rate=1.0`` (every rally hidden), whereas bounding it
into pieces
213         that each glue 2 neighbours is ``merge_count=9`` yet ``merge_rate=0.5`` (FEWER
```

```
rallies hidden).
        So the strict count rule blocks a config that strictly *reduces* the fraction of
rallies lost to
        over-merge. (Measured: the cap6+R4 milestone trips strict on 10/11 videos yet cuts
mean
        ``merge_rate`` 0.694 ? 0.150 with NO per-video merge_rate regression ?
RALLY_QUALITY_RESEARCH §6.)

        **The net rule (default verdict ``passes``):** score over-seg as ONE weighted cost
per video and
        require BOTH:
          1. the corpus-mean net cost does not rise (``cand_net ? base_net + eps``), AND
          2. NO video's **merge_rate** rises beyond ``eps`` ? reintroducing a mega-window
that hides MORE
             of a video's rallies is the one over-merge regression that is never acceptable,
the honest
             "no NEW merges" rule expressed on the *rate* (the recall-meaningful quantity),
not the count.
        With ``use_rates`` the per-video cost uses ``merge_rate``/``split_rate`` (fraction
of GT rallies
        affected, ? [0,1] ? comparable across a structural change); ``use_rates=False``
scores raw counts.

        **Revert (one line):** read ``strict_passes`` instead of ``passes`` to fall back to
the original
        strict per-video count rule. Both verdicts are always reported, so the choice is
auditable.
        """

        def _aligned() -> List[Tuple[Dict[str, Any], Dict[str, Any]]]:
            if (
                base
                and cand
                and all("name" in b for b in base)
                and all("name" in c for c in cand)
            ):
                cmap = {c["name"]: c for c in cand}
                return [(b, cmap[b["name"]]) for b in base if b["name"] in cmap]
            n = min(len(base), len(cand))
            return [(base[i], cand[i]) for i in range(n)]

        def _cost(row: Dict[str, Any]) -> float:
            if use_rates:
                return weight_merge * float(row["merge_rate"]) + weight_split * float(
                    row["split_rate"]
                )
            return weight_merge * float(row["merge_count"]) + weight_split * float(
                row["split_count"]
            )

        pairs = _aligned()
        n = len(pairs)
        base_costs = [_cost(b) for b, _ in pairs]
        cand_costs = [_cost(c) for _, c in pairs]
        base_net = (sum(base_costs) / n) if n else 0.0
        cand_net = (sum(cand_costs) / n) if n else 0.0
        net_delta = cand_net - base_net

        # Per-video regressions, reported for transparency.
        new_merges: List[Dict[str, Any]] = []  # raw COUNT increases (what strict trips on)
        new_splits: List[Dict[str, Any]] = []  # raw COUNT increases (what strict trips on)
        merge_rate_regress: List[
            Dict[str, Any]
        ] = []  # RATE increases (the real over-merge regression)
        for b, c in pairs:
```

```
            if float(c["merge_count"]) > float(b["merge_count"]) + eps:
                new_merges.append(
                    {
                        "name": b.get("name", ""),
                        "base": float(b["merge_count"]),
                        "cand": float(c["merge_count"]),
                    }
                )
            if float(c["split_count"]) > float(b["split_count"]) + eps:
                new_splits.append(
                    {
                        "name": b.get("name", ""),
                        "base": float(b["split_count"]),
                        "cand": float(c["split_count"]),
                    }
                )
            if use_rates and float(c["merge_rate"]) > float(b["merge_rate"]) + eps:
                merge_rate_regress.append(
                    {
                        "name": b.get("name", ""),
                        "base": float(b["merge_rate"]),
                        "cand": float(c["merge_rate"]),
                    }
                )

        strict_passes = (not new_merges) and (not new_splits)
        net_cost_ok = net_delta <= eps
        no_merge_rate_regress = not merge_rate_regress
        passes = bool(net_cost_ok and no_merge_rate_regress)
        return {
            "passes": passes,
            "strict_passes": strict_passes,
            "net_cost_ok": net_cost_ok,
            "no_merge_rate_regress": no_merge_rate_regress,
            "base_net": base_net,
            "cand_net": cand_net,
            "net_delta": net_delta,
            "n": n,
            "use_rates": use_rates,
            "weight_merge": weight_merge,
            "weight_split": weight_split,
            "new_merges": new_merges,
            "new_splits": new_splits,
            "merge_rate_regress": merge_rate_regress,
        }


    def tolerance_report(
        preds: List[Interval],
        gts: List[Interval],
        tau_start: float = TAU_START,
        tau_end: float = TAU_END,
    ) -> Dict[str, object]:
        """The full R2 block for one video: P/R/F1@? + boundary-error distributions + over-
seg guard."""
        matches, precision, recall, f1 = match_1to1(preds, gts, tau_start, tau_end)
        return {
            "tau_start": tau_start,
            "tau_end": tau_end,
            "n_pred": len(preds),
            "n_gt": len(gts),
            "matched": len(matches),
            "precision": precision,
            "recall": recall,
            "f1": f1,
```

```
331             "boundary_error": boundary_error_report(preds, gts),
332             "over_seg": over_seg_report(preds, gts),
333         }
334
335
336     def tolerance_sweep(
337         preds: List[Interval], gts: List[Interval], taus: Tuple[float, ...] = SWEEP_TAUS
338     ) -> Dict[float, float]:
339         """Symmetric-? F1 sweep ? the trend curve (a single ? misleads). Monotonic non-
decreasing in ?."""
340         return {t: match_1to1(preds, gts, t, t)[3] for t in taus}
341
```

6. user (2026-07-06T20:52:57.076Z)

```
1       """Canonical ground-truth interval loader ? THE one GT/golden-CSV reader (ADR-008
prereq).
2
3       The repo historically carried two incompatible loaders:
4       - ``annotations.load_annotations``: positional ``start,end`` (the collector's
5         ``curate export`` format), raises on bad rows;
6       - ``calibrate_wasb.load_golden_gts``: DictReader requiring ``start_time/end_time``
7         (the rally-annotator golden format), silently skipping bad rows ? fed 7 modules.
8
9       A file in one convention fed to the other loader yielded zero/garbage intervals with no
10      error. This module accepts BOTH conventions (header-mapped, else positional), so every
11      quantity the ship-gate depends on (gt_hash, LOVO scores, the future consent fingerprint)
12      flows through one reader. Both legacy entry points now delegate here.
13
14      Accepted shapes (one rally per row; blank lines and ``#`` comments ignored):
15      - header with ``start``/``end`` OR ``start_time``/``end_time`` columns (any other
16        columns ? rally_number, ending_reason, sport, ? ? are ignored);
17      - no header: columns 0,1 positionally.
18      Timestamps: decimal seconds or ``MM:SS`` / ``HH:MM:SS``
(``annotations.parse_timestamp``).
19      """
20
21      import csv
22      import logging
23      from typing import List, Optional, Tuple
24
25      from backend.eval.annotations import parse_timestamp
26
27      logger = logging.getLogger(__name__)
28
29      Interval = Tuple[float, float]
30
31      _START_NAMES = ("start", "start_time")
32      _END_NAMES = ("end", "end_time")
33
34
35      def _header_columns(row: List[str]) -> Optional[Tuple[int, int]]:
36          """If `row` is a header naming start/end columns, return their indices, else
None."""
37          cells = [(c or "").strip().lower() for c in row]
38          start_idx = next((i for i, c in enumerate(cells) if c in _START_NAMES), None)
39          end_idx = next((i for i, c in enumerate(cells) if c in _END_NAMES), None)
40          if start_idx is not None and end_idx is not None:
41              return start_idx, end_idx
42          return None
43
44
45      def load_gt_intervals(csv_path: str, *, strict: bool = True) -> List[Interval]:
46          """Load GT rally intervals from either golden-CSV convention.
47
```

```
        strict=True (corpus/gate use): raise ValueError with file:line on any malformed row
        (bad timestamp, end <= start, negative start) so labeling mistakes fail loudly.
        strict=False (legacy tuning-driver behavior): skip bad rows, but LOG how many were
        skipped ? a silently shrinking GT set flatters precision.

        Returns intervals sorted by start; overlapping GT intervals are logged as a warning
        (they break one-to-one matching and usually indicate a labeling error).
        """
        intervals: List[Interval] = []
        skipped = 0
        cols: Optional[Tuple[int, int]] = None  # (start_idx, end_idx); None until detected
        saw_header = False

        with open(csv_path, newline="", encoding="utf-8-sig") as f:
            for lineno, row in enumerate(csv.reader(f), start=1):
                if not row or not (row[0] or "").strip() or row[0].strip().startswith("#"):
                    continue
                if cols is None and not saw_header:
                    header = _header_columns(row)
                    if header is not None:
                        cols = header
                        saw_header = True
                        continue  # consume the header row
                    cols = (0, 1)  # headerless: positional start,end
                assert cols is not None  # every branch above assigns it
                si, ei = cols
                try:
                    if len(row) <= max(si, ei):
                        raise ValueError(
                            f"expected at least {max(si, ei) + 1} columns, got {row!r}"
                        )
                    start = parse_timestamp(row[si])
                    end = parse_timestamp(row[ei])
                    if start < 0:
                        raise ValueError(f"negative start ({start})")
                    if start >= end:
                        raise ValueError(f"start ({start}) >= end ({end})")
                except ValueError as e:
                    if strict:
                        raise ValueError(f"{csv_path}:{lineno}: {e}") from e
                    skipped += 1
                    continue
                intervals.append((start, end))

        if skipped:
            logger.warning(
                "%s: skipped %d malformed GT row(s) (strict=False)", csv_path, skipped
            )

        intervals.sort(key=lambda iv: iv[0])
        for (s1, e1), (s2, e2) in zip(intervals, intervals[1:]):
            if s2 < e1:
                logger.warning(
                    "%s: overlapping GT intervals (%.3f-%.3f vs %.3f-%.3f) ? "
                    "labeling error? Overlaps break one-to-one matching.",
                    csv_path,
                    s1,
                    e1,
                    s2,
                    e2,
                )
        return intervals

```

7. user (2026-07-06T20:52:57.122Z)

```
"""One-command rally-segmentation eval ? score a windowing against the golden labels with the
    over-segmentation-sensitive metric set, honestly.

    This is the harness the rally-quality project (``docs/RALLY_QUALITY_RESEARCH.md`` §4.5 / §7)
    measures every tier against. It closes the loop the over-merge finding exposed: plain temporal
    IoU-F1 is blind to merged mega-windows, so we report it **alongside** segmental F1@k, the
    segment-count ratio, and the explicit ``merge_split`` breakdown (``segmentation_metrics``).

    For each golden video it: parses the cached WASB/TrackNet trajectory CSV ? builds candidate
    windows under a :class:`~backend.eval.windowing.WindowingPreset` (so it tracks the SERVED
    operating point and any future windowing the same way) ? scores the windows-as-rallies against
    the ground-truth rally intervals. Aggregates per-video held-out F1 with a bootstrap CI, and can
    diff two windowings with a paired sign-test + CI (the per-tier ``delta``).

    GROUND TRUTH: the golden manifest (``output/human_lovo_manifest.json``) gives each video's
    trajectory path + fps + frame_width; the GT rally intervals come from
    ``output/<name>_golden_features.json`` (the ``gts`` key) so this needs no extra label files.

    Pure offline (CPU): trajectory parsing + windowing + interval scoring; no GPU/Gemini.
    """

    from __future__ import annotations

    import argparse
    import json
    import os
    from typing import Any, Dict, List, Optional, Tuple

    from backend.eval import eval_stats, metrics, segmentation_metrics, windowing
    from backend.eval.golden_manifest import DEFAULT_MANIFEST, load_resolved_manifest
    from backend.eval.metrics import Interval
    from backend.eval.windowing import PRESETS, SERVED, WindowingPreset

    DEFAULT_FEATURES_DIR = "output"
    DEFAULT_IOU = 0.5

    #: Long-rally lens threshold (s). A rally is "long" (an eye-catching highlight) when its duration
    #: (end ? start) exceeds this. The local detector over-fires, and its false positives are mostly
    #: SHORT (motion blips, background-court fragments on multicourt footage); filtering to long rallies
    #: strips most of those FPs and reveals the real product quality on the long-rally happy path. Used
    #: as the default split point for the stratified report (``--long-tau`` overrides it). DURATION, not
    #: shot-count, is the filter: golden ``shots_count`` is unlabeled and the no-AI path reports it as
    #: "Unknown", whereas duration is derived from the same start/end labels we already trust.
    LONG_RALLY_TAU = 5.0

    #: Multicourt golden clips ? the court-type lookup for the stratified report. The golden manifest
```

```
47      #: (``output/human_lovo_manifest.json``) is gitignored, so this committed set is the
AUDITABLE source
48      #: of the single-vs-multicourt strata, kept in sync with ``docs/GOLDEN_VIDEOS.md`` (the
clips tagged
49      #: "multicourt"). Multicourt footage carries background games on adjacent courts ? the
dominant
50      #: source of short false positives ? so we report it separately. A manifest entry MAY
carry an
51      #: explicit ``court_type`` field, which overrides this set (see :func:`court_type_for`).
52      MULTICOURT_CLIPS = frozenset(
53          {"mahadevpura_2", "GX010128", "Badminton_BXH_2", "Boxhill_Doubles"}
54      )
55
56
57      def court_type_for(video: Dict[str, Any]) -> str:
58          """``'multicourt'`` or ``'single'`` for a golden video. Honors an explicit manifest
59          ``court_type`` field when present, else falls back to membership in
``MULTICOURT_CLIPS``
60          (keyed by ``name``), else ``'single'``."""
61          ct = video.get("court_type")
62          if isinstance(ct, str) and ct:
63              return ct
64          return "multicourt" if video.get("name") in MULTICOURT_CLIPS else "single"
65
66
67      def filter_by_min_duration(ivs: List[Interval], min_duration: float) -> List[Interval]:
68          """The long-rally lens: keep only intervals strictly longer than ``min_duration``
seconds.
69
70          ``min_duration <= 0`` is an EXACT passthrough (the default-OFF semantics ? every
real rally has
71          positive duration, so nothing is dropped). It's a FILTER, not a new metric: applied
identically
72          to predictions AND ground truth before any matching, so the tIoU and R2 paths stay
consistent."""
73          if min_duration <= 0.0:
74              return list(ivs)
75          return [iv for iv in ivs if (iv[1] - iv[0]) > min_duration]
76
77
78      # ---------------------------------------------------------------------------- #
79      # Pure scoring core (no I/O ? unit-testable)
80      # ---------------------------------------------------------------------------- #
81      def score_intervals(
82          preds: List[Interval],
83          gts: List[Interval],
84          iou: float = DEFAULT_IOU,
85          tau_start: Optional[float] = None,
86          tau_end: Optional[float] = None,
87          min_duration: float = 0.0,
88      ) -> Dict[str, Any]:
89          """The full per-video metric row for predicted vs ground-truth rally intervals.
90
91          Pairs temporal-IoU P/R/F1 + boundary error (``metrics.score``) with the
92          over-segmentation-sensitive set (F1@k, segment-count ratio, merge/split breakdown)
so a
93          merged mega-window is *visible* even when IoU-F1 isn't moved.
94
95          Also carries the **R2** task-faithful block (``tolerance_metrics``, the headline
going forward;
96          tIoU here is the SECONDARY trend-line per the augment-then-ablation-gate decision):
strict-1:1
97          tolerance-match ``tol_f1/tol_precision/tol_recall`` (over-production costs
precision) + the
98          start/end boundary-error medians + the over-seg guard counts. See
```

```
docs/R2_EVAL_METRIC_DESIGN.md.
99
100         ``min_duration`` > 0 applies the **long-rally lens**
(:func:`filter_by_min_duration`): BOTH preds
101         and GTs are filtered to rallies longer than that many seconds BEFORE any matching,
so the whole
102         row (tIoU set AND R2 block) scores only long rallies. Default 0.0 = OFF (exact
passthrough).
103         """
104         from backend.eval import tolerance_metrics as tm
105
106         preds = filter_by_min_duration(preds, min_duration)
107         gts = filter_by_min_duration(gts, min_duration)
108         ts = tm.TAU_START if tau_start is None else tau_start
109         te = tm.TAU_END if tau_end is None else tau_end
110         sc = metrics.score(preds, gts, iou_threshold=iou)
111         f1k = segmentation_metrics.f1_at_overlaps(preds, gts)
112         ms = segmentation_metrics.merge_split_report(preds, gts)
113         tol_matches, tol_p, tol_r, tol_f1 = tm.match_1to1(preds, gts, ts, te)
114         be = tm.boundary_error_report(
115             preds, gts
116         )  # unconditional best-overlap (the R4-aiming diagnostic)
117         osr = tm.over_seg_report(preds, gts)
118         return {
119             "f1": sc.f1,
120             "precision": sc.precision,
121             "recall": sc.recall,
122             "mean_iou": sc.mean_iou,
123             "mean_start_error": sc.mean_start_error,
124             "mean_end_error": sc.mean_end_error,
125             "f1@0.1": f1k[0.1],
126             "f1@0.25": f1k[0.25],
127             "f1@0.5": f1k[0.5],
128             "segment_count_ratio": segmentation_metrics.segment_count_ratio(preds, gts),
129             "merge_rate": ms["merge_rate"],
130             "split_rate": ms["split_rate"],
131             "merges": ms["merges"],
132             "splits": ms["splits"],
133             "missed": ms["missed"],
134             "spurious": ms["spurious"],
135             "num_pred": len(preds),
136             "num_gt": len(gts),
137             # --- R2 (tolerance-match) ? the task-faithful headline block ---
138             "tau_start": ts,
139             "tau_end": te,
140             "tol_f1": tol_f1,
141             "tol_precision": tol_p,
142             "tol_recall": tol_r,
143             "tol_matched": float(len(tol_matches)),
144             "dstart_abs_median": be["dstart"]["abs_median"],
145             "dend_abs_median": be["dend"]["abs_median"],
146             "split_count": osr["split_count"],
147             "merge_count": osr["merge_count"],
148         }
149
150
151     _AGG_KEYS = (
152         "f1",
153         "f1@0.25",
154         "f1@0.5",
155         "merge_rate",
156         "split_rate",
157         "segment_count_ratio",
158         "precision",
159         "recall",
```

```
160         "tol_f1",
161         "tol_precision",
162         "tol_recall",
163         "dstart_abs_median",
164         "dend_abs_median",
165     )
166
167
168     def aggregate(rows: List[Dict[str, Any]]) -> Dict[str, Any]:
169         """Aggregate per-video rows: mean (+ bootstrap 95% CI) of the headline metrics
across videos.
170
171         Per-video means (not pooled) keep videos the unit of evidence, matching the
harness's
172         leave-one-video-out discipline (windows within a video are correlated)."""
173         out: Dict[str, Any] = {"n": len(rows)}
174         for k in _AGG_KEYS:
175             vals = [r[k] for r in rows]
176             out[k] = eval_stats.mean_with_ci(vals) if vals else None
177         return out
178
179
180     # ------------------------------------------------------------------------ #
181     # Golden corpus I/O
182     # ------------------------------------------------------------------------ #
183     def load_golden(
184         manifest_path: str = DEFAULT_MANIFEST, features_dir: str = DEFAULT_FEATURES_DIR
185     ) -> List[Dict[str, Any]]:
186         """Load each golden video's trajectory path + fps + frame_width (manifest) and GT
rally
187         intervals (``<name>_golden_features.json`` ``gts``). Videos with no GTs are kept but
flagged."""
188         manifest = load_resolved_manifest(manifest_path, required_fields=("trajectory",))
189         out: List[Dict[str, Any]] = []
190         for v in manifest:
191             feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")
192             gts: List[Interval] = []
193             if os.path.exists(feat):
194                 with open(feat) as fh:
195                     gts = [
196                         (float(s), float(e)) for s, e in (json.load(fh).get("gts") or [])
197                     ]
198             entry: Dict[str, Any] = {
199                 "name": v["name"],
200                 "trajectory": v["trajectory"],
201                 "fps": float(v["fps"]),
202                 "frame_width": float(v["frame_width"]),
203                 "gts": gts,
204             }
205             # Optional prominent-court gate inputs (multicourt G2; default absent ? the gate
is a no-op):
206             # a normalized court_polygon + optional frame_height + court_height_pad from the
manifest.
207             for k in ("court_polygon", "frame_height", "court_height_pad"):
208                 if k in v:
209                     entry[k] = v[k]
210             out.append(entry)
211         return out
212
213
214     def windows_for(
215         video: Dict[str, Any], preset: WindowingPreset, min_crossings: int = 0
216     ) -> List[Interval]:
217         """Parse a video's trajectory, window it under ``preset`` ? predicted rally
intervals.
```

```
218
219         ``min_crossings`` > 0 applies the **net-crossings rally-state gate** (Gate-A,
220         ``rally_gate.gate_windows``): drop windows whose shuttle crosses the net fewer than
this many
221         times (a non-rally fragment). This is the cheapest rally-state cue (W2) ? CPU-only,
derived
222         from the trajectory + an auto-estimated net axis; it cleans up the over-split that
bounded
223         windowing (W1) leaves behind.
224
225         NEVER-EMPTY safeguard (mirrors ``FusionSegmenter._select_for_handoff``): if the gate
would
226         drop EVERY window of a video that had ?1, fall back to the ungated windows. The gate
is
227         strictly *pruning* ? it never zeroes out a non-empty video ? so W2-on-W1 is "do no
harm".
228         """
229         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
230
231         points = TrackNetRunner.parse_trajectory_csv(video["trajectory"])
232         wins = windowing.build_windows(points, video["fps"], video["frame_width"], preset)
233         ivs = windowing.windows_to_intervals(wins)
234         # Prominent-court WINDOW gate (multicourt G2, default-OFF): when the video carries a
normalized
235         # court_polygon, drop candidate rallies whose shuttle is MAJORITY outside the
prominent court ? a
236         # background/adjacent-court rally. Window-level (not point-level): a background
rally's central
237         # tail keeps a few in-court points, enough to hold a window, so point-clearing alone
is too leaky
238         # (measured on GX010142 rally #1). No polygon ? no-op. See
docs/PROMINENT_COURT_DETECTION.md.
239         poly = video.get("court_polygon")
240         if poly and ivs:
241             from backend.pipeline.detectors.court_gate import gate_intervals_to_court
242
243             fw = float(video["frame_width"])
244             fh = float(video.get("frame_height") or fw * 9.0 / 16.0)
245             ivs = gate_intervals_to_court(
246                 ivs,
247                 points,
248                 poly,
249                 float(video["fps"]),
250                 fw,
251                 fh,
252                 min_in_court_frac=float(video.get("court_min_in_court_frac", 0.5)),
253                 height_pad=float(video.get("court_height_pad", 0.0) or 0.0),
254             )
255         if min_crossings > 0 and ivs:
256             from backend.pipeline.detectors.rally_gate import gate_windows
257
258             gated = [
259                 (g.start, g.end)
260                 for g in gate_windows(
261                     points, ivs, video["fps"], min_crossings=min_crossings
262                 )
263                 if g.kept
264             ]
265             # Gating emptied a non-empty video ? keep the ungated windows.
266             ivs = gated if gated else ivs
267         return ivs
268
269
270     #: One windowed video before scoring: (name, court_type, predicted intervals, GT
intervals).
```

```
271     PredictedVideo = Tuple[str, str, List[Interval], List[Interval]]
272
273
274     def predict_all(
275         golden: List[Dict[str, Any]], preset: WindowingPreset, min_crossings: int = 0
276     ) -> List["PredictedVideo"]:
277         """Window every scorable golden video ONCE (the expensive trajectory-parse +
windowing step),
278         returning ``(name, court_type, preds, gts)`` per video. Lets callers re-score the
same
279         predictions under several lenses (e.g. the all-vs-long stratified report) without
re-windowing."""
280         out: List[PredictedVideo] = []
281         for v in golden:
282             if not v["gts"] or not os.path.exists(v["trajectory"]):
283                 continue
284             out.append(
285                 (
286                     v["name"],
287                     court_type_for(v),
288                     windows_for(v, preset, min_crossings),
289                     v["gts"],
290                 )
291             )
292         return out
293
294
295     def _score_predicted(
296         predicted: List["PredictedVideo"],
297         iou: float,
298         tau_start: Optional[float],
299         tau_end: Optional[float],
300         min_duration: float,
301     ) -> List[Dict[str, Any]]:
302         """Score pre-windowed videos into per-video rows (carrying ``name`` +
``court_type``)."""
303         rows: List[Dict[str, Any]] = []
304         for name, court, preds, gts in predicted:
305             row = score_intervals(preds, gts, iou, tau_start, tau_end, min_duration)
306             row["name"] = name
307             row["court_type"] = court
308             rows.append(row)
309         return rows
310
311
312     def evaluate(
313         golden: List[Dict[str, Any]],
314         preset: WindowingPreset,
315         iou: float = DEFAULT_IOU,
316         min_crossings: int = 0,
317         tau_start: Optional[float] = None,
318         tau_end: Optional[float] = None,
319         min_duration: float = 0.0,
320     ) -> Dict[str, Any]:
321         """Score every golden video under ``preset`` (+ optional net-crossings gate); return
per-video
322         rows + the aggregate. Each row carries both the tIoU set and the R2 tolerance-match
block, plus
323         its ``court_type``. ``min_duration`` > 0 applies the long-rally lens (preds + GTs
filtered to
324         rallies longer than that many seconds) consistently across both metric paths."""
325         rows = _score_predicted(
326             predict_all(golden, preset, min_crossings),
327             iou,
328             tau_start,
```

```
            tau_end,
            min_duration,
        )
        return {
            "preset": preset.to_dict(),
            "iou": iou,
            "min_crossings": min_crossings,
            "min_duration": min_duration,
            "rows": rows,
            "aggregate": aggregate(rows),
        }


    def compare(
        golden: List[Dict[str, Any]],
        base: WindowingPreset,
        cand: WindowingPreset,
        iou: float = DEFAULT_IOU,
        base_min_crossings: int = 0,
        cand_min_crossings: int = 0,
        tau_start: Optional[float] = None,
        tau_end: Optional[float] = None,
        guard_weight_merge: Optional[float] = None,
        guard_weight_split: Optional[float] = None,
        guard_use_rates: bool = True,
        min_duration: float = 0.0,
    ) -> Dict[str, Any]:
        """Baseline vs candidate (windowing and/or net-crossings gate): per-video paired ?
(+ CI +
        sign-test) on both tIoU-F1 and the R2 ``tol_f1`` ? the per-tier 'number'. Also
reports ? on the
        over-segmentation metrics AND the **R1 net over-seg promotion guard**
(``over_seg_guard``):
        the honest promote/block verdict for cand-over-base, with the strict per-video rule
reported
        alongside so the refinement is auditable. ``min_duration`` > 0 scores both sides
through the
        long-rally lens (same filter applied to base and cand ? it's a scoring lens, not a
windowing
        knob, so it is identical on both sides of the A/B)."""
        from backend.eval import tolerance_metrics as tm

        b = evaluate(
            golden, base, iou, base_min_crossings, tau_start, tau_end, min_duration
        )
        c = evaluate(
            golden, cand, iou, cand_min_crossings, tau_start, tau_end, min_duration
        )
        by_name_b = {r["name"]: r for r in b["rows"]}
        paired = [(by_name_b[r["name"]], r) for r in c["rows"] if r["name"] in by_name_b]
        delta: Dict[str, Any] = {}
        for k in (
            "f1",
            "f1@0.25",
            "tol_f1",
            "merge_rate",
            "split_rate",
            "segment_count_ratio",
        ):
            delta[k] = eval_stats.paired_delta_ci(
                [rb[k] for rb, _ in paired], [rc[k] for _, rc in paired]
            )
        gkw: Dict[str, Any] = {"use_rates": guard_use_rates}
        if guard_weight_merge is not None:
            gkw["weight_merge"] = guard_weight_merge
```

```
        if guard_weight_split is not None:
            gkw["weight_split"] = guard_weight_split
        guard = tm.over_seg_guard([rb for rb, _ in paired], [rc for _, rc in paired], **gkw)
        return {
            "base": b,
            "cand": c,
            "n_paired": len(paired),
            "delta": delta,
            "over_seg_guard": guard,
        }


    def stratified_report(
        golden: List[Dict[str, Any]],
        preset: WindowingPreset,
        iou: float = DEFAULT_IOU,
        min_crossings: int = 0,
        tau_start: Optional[float] = None,
        tau_end: Optional[float] = None,
        long_tau: float = LONG_RALLY_TAU,
    ) -> Dict[str, Any]:
        """The long-rally lens payoff: ``{all vs long(>long_tau)} × {single vs multicourt}``
aggregates.

        The thesis: local OVER-FIRES, and its false positives are mostly SHORT (motion
blips,
        background-court fragments on multicourt footage). Filtering to long rallies should
strip those
        FPs ? precision (``tol_precision``) should jump, most on multicourt. This table is
how we read
        "do we beat Gemini on the single-match long-rally happy path?".

        Windowing is run ONCE per video (``predict_all``); each cell re-scores the same
predictions
        under its duration lens. A cell aggregates only videos with >=1 golden rally in that
stratum
        (``num_gt`` > 0): a clip with no long rally is not evidence about long-rally quality
and its
        0/0 recall would distort the mean. ``n`` per cell reports the videos that survived
that gate."""
        predicted = predict_all(golden, preset, min_crossings)
        long_label = f"long>{long_tau:g}s"
        strata = (("all", 0.0), (long_label, long_tau))
        courts = ("single", "multicourt", "all")
        cells: Dict[Tuple[str, str], Dict[str, Any]] = {}
        for dur_label, min_dur in strata:
            rows = _score_predicted(predicted, iou, tau_start, tau_end, min_dur)
            for court in courts:
                sub = [
                    r
                    for r in rows
                    if (court == "all" or r["court_type"] == court) and r["num_gt"] > 0
                ]
                cells[(dur_label, court)] = aggregate(sub)
        return {
            "long_tau": long_tau,
            "labels": [s[0] for s in strata],
            "courts": list(courts),
            "cells": cells,
            "preset": preset.to_dict(),
            "iou": iou,
            "min_crossings": min_crossings,
        }


```

```
    # ---------------------------------------------------------------------- #
    # Reporting
    # ---------------------------------------------------------------------- #
    def _ci(m: Optional[Dict[str, Any]]) -> str:
        return "?" if not m else f"{m['mean']:.3f} [{m['ci_low']:.3f},{m['ci_high']:.3f}]"


    def _format_guard(g: Dict[str, Any]) -> str:
        """Render the R1 net over-seg promotion-guard verdict (with the strict rule for
contrast)."""
        basis = "rate" if g["use_rates"] else "count"
        lines = [
            f"\n## R1 over-seg promotion guard ({basis}-based, "
            f"w_merge={g['weight_merge']}, w_split={g['weight_split']})",
            f"  NET verdict: {'PASS' if g['passes'] else 'BLOCK'}  "
            f"(net cost {g['base_net']:.3f} ? {g['cand_net']:.3f}, ? {g['net_delta']:+.3f};
"
            f"no_merge_rate_regress={g['no_merge_rate_regress']})",
            f"  strict per-video rule (R2 §2.3.3, for contrast): "
            f"{'pass' if g['strict_passes'] else 'BLOCK'}  "
            f"(count increases ? merges:{len(g['new_merges'])}
splits:{len(g['new_splits'])})",
        ]
        if g["merge_rate_regress"]:
            names = ", ".join(
                f"{r['name']}({r['base']:.2f}?{r['cand']:.2f})"
                for r in g["merge_rate_regress"]
            )
            lines.append(f"  ? merge_rate REGRESSED on: {names}")
        return "\n".join(lines)


    def format_report(result: Dict[str, Any]) -> str:
        p = result["preset"]
        bounded = []
        if p.get("max_window_duration"):
            bounded.append(
                f"cap={p['max_window_duration']}" + ("(hard)" if p.get("hard_cap") else "")
            )
        if p.get("inactivity_end"):
            bounded.append(
                f"inact_end={p['inactivity_end']}/rt={p['inactivity_rally_thresh']}"
                f"/md={p['inactivity_min_density']}"
            )
        if p.get("blob_fallback"):
            bounded.append("blob_fallback")
        extra = (", " + ", ".join(bounded)) if bounded else ""
        md = result.get("min_duration", 0.0) or 0.0
        md_tag = f", LONG-RALLY LENS min_dur>{md:g}s" if md > 0 else ""
        lines = [
            f"# Rally-segmentation eval ? windowing '{p['name']}' "
            f"(vt={p['velocity_thresh']}, merge_gap={p['merge_gap']}, "
            f"min_win={p['min_window_duration']}{extra}), IoU={result['iou']}{md_tag}",
            "",
        ]
        lines.append(
            "| video | F1 | F1@0.25 | F1@0.5 | merges | merge_rate | seg_ratio | pred/gt |"
        )
        lines.append("|---|---|---|---|---|---|---|---|")
        for r in result["rows"]:
            lines.append(
                f"| {r['name']} | {r['f1']:.3f} | {r['f1@0.25']:.3f} | {r['f1@0.5']:.3f} | "
                f"{int(r['merges'])} | {r['merge_rate']:.2f} |
{r['segment_count_ratio']:.2f} | "
                f"{r['num_pred']}/{r['num_gt']} |"
```

```
            )
        a = result["aggregate"]
        lines += [
            "",
            f"**Aggregate (n={a['n']}, mean [95% CI]):** "
            f"F1 {_ci(a['f1'])} · F1@0.25 {_ci(a['f1@0.25'])} · F1@0.5 {_ci(a['f1@0.5'])} ·
"
            f"merge_rate {_ci(a['merge_rate'])} · seg_ratio
{_ci(a['segment_count_ratio'])}",
        ]
        # --- R2 (tolerance-match) block ? the task-faithful headline (tIoU above is
secondary) ---
        rows = result["rows"]
        if rows and "tol_f1" in rows[0]:
            ts, te = rows[0]["tau_start"], rows[0]["tau_end"]
            lines += [
                "",
                f"## R2 ? tolerance-match (?_start={ts}s, ?_end={te}s, strict 1:1)",
                "",
                "| video | tolF1 | tolP | tolR | |?start| | |?end| | split | merge |",
                "|---|---|---|---|---|---|---|---|",
            ]
            for r in rows:
                lines.append(
                    f"| {r['name']} | {r['tol_f1']:.3f} | {r['tol_precision']:.2f} | "
                    f"{r['tol_recall']:.2f} | {r['dstart_abs_median']:.2f} | "
                    f"{r['dend_abs_median']:.2f} | {int(r['split_count'])} |
{int(r['merge_count'])} |"
                )
            lines += [
                "",
                f"**R2 aggregate (n={a['n']}):** tolF1 {_ci(a['tol_f1'])} · "
                f"tolP {_ci(a['tol_precision'])} · tolR {_ci(a['tol_recall'])} · "
                f"|?start| {_ci(a['dstart_abs_median'])} · |?end|
{_ci(a['dend_abs_median'])} "
                f"(start?solved / end=the gap)",
            ]
        return "\n".join(lines)


    def format_stratified(strat: Dict[str, Any]) -> str:
        """Render the long-rally stratified table: {all vs long} × {single, multicourt, all-
courts}.

        ``tolP`` is the column that carries the story ? if filtering to long rallies strips
local's
        short false positives, precision rises (most on multicourt). ``n`` is the videos
with >=1 golden
        rally in that stratum."""
        long_tau = strat["long_tau"]
        cells = strat["cells"]
        lines = [
            f"\n## Long-rally lens ? stratified (long = duration > {long_tau:g}s; "
            "n = videos with >=1 golden rally in the stratum)",
            "",
            "| stratum | court | n | tolF1 | tolP | tolR | tIoU F1@0.5 | seg_ratio |",
            "|---|---|---|---|---|---|---|---|",
        ]
        for dur_label in strat["labels"]:
            for court in strat["courts"]:
                a = cells[(dur_label, court)]
                lines.append(
                    f"| {dur_label} | {court} | {a['n']} | {_ci(a['tol_f1'])} | "
                    f"{_ci(a['tol_precision'])} | {_ci(a['tol_recall'])} | "
                    f"{_ci(a['f1@0.5'])} | {_ci(a['segment_count_ratio'])} |"
```

```
                )
        return "\n".join(lines)


    def _resolve_preset(name: str) -> WindowingPreset:
        if name in PRESETS:
            return PRESETS[name]
        raise SystemExit(f"unknown preset '{name}'; choices: {sorted(PRESETS)}")


    def _build_preset(args: argparse.Namespace, side: str) -> WindowingPreset:
        """Resolve the ``base`` (``--preset``) or ``cand`` (``--vs``) windowing from
``args`` by
        applying the ``dataclasses.replace`` overrides for cap / hard-cap / blob-fallback /
        inactivity. ``cand`` falls back to the base override when its ``--vs-*`` variant is
unset
        (so a baseline-vs-candidate promotion check runs in one shot)."""
        from dataclasses import replace

        if side == "base":
            preset = _resolve_preset(args.preset)
            cap = args.max_window_duration
            hard_cap = args.hard_cap
            blob_fallback = args.blob_fallback
            inactivity_end = args.inactivity_end
            temporal_density = args.inactivity_temporal_density
        else:
            preset = _resolve_preset(args.vs)
            cap = (
                args.vs_max_window_duration
                if args.vs_max_window_duration is not None
                else args.max_window_duration
            )
            hard_cap = args.vs_hard_cap or args.hard_cap
            blob_fallback = args.vs_blob_fallback or args.blob_fallback
            inactivity_end = (
                args.vs_inactivity_end
                if args.vs_inactivity_end is not None
                else args.inactivity_end
            )
            temporal_density = (
                args.vs_inactivity_temporal_density or args.inactivity_temporal_density
            )
        if cap is not None:
            preset = replace(preset, max_window_duration=cap)
        if hard_cap:
            preset = replace(preset, hard_cap=True)
        if blob_fallback:
            preset = replace(preset, blob_fallback=True)
        if temporal_density:
            preset = replace(preset, inactivity_temporal_density=True)
        if args.blob_factor is not None:
            preset = replace(preset, blob_factor=args.blob_factor)
        if inactivity_end is not None:
            preset = replace(preset, inactivity_end=inactivity_end)
        if args.inactivity_rally_thresh is not None:
            preset = replace(preset, inactivity_rally_thresh=args.inactivity_rally_thresh)
        if args.inactivity_min_density is not None:
            preset = replace(preset, inactivity_min_density=args.inactivity_min_density)
        return preset


    def main() -> None:
        ap = argparse.ArgumentParser(
            description="Rally-segmentation eval vs golden labels (offline)."
```

```
        )
        ap.add_argument("--manifest", default=DEFAULT_MANIFEST)
        ap.add_argument("--features-dir", default=DEFAULT_FEATURES_DIR)
        ap.add_argument(
            "--preset", default=SERVED.name, help="windowing preset (default: served)"
        )
        ap.add_argument("--vs", default=None, help="second preset to diff against --preset")
        ap.add_argument("--iou", type=float, default=DEFAULT_IOU)
        ap.add_argument(
            "--min-crossings",
            type=int,
            default=0,
            help="net-crossings rally-state gate on --preset (0=off; W2 Gate-A)",
        )
        ap.add_argument(
            "--vs-min-crossings",
            type=int,
            default=0,
            help="net-crossings gate on --vs (default: same as --min-crossings)",
        )
        ap.add_argument(
            "--max-window-duration",
            type=float,
            default=None,
            help="override bounded-windowing cap (s) on BOTH presets (W1; e.g. 12)",
        )
        ap.add_argument(
            "--vs-max-window-duration",
            type=float,
            default=None,
            help="cap (s) on --vs ONLY (default: same as --max-window-duration); lets a "
            "BASELINE (no cap) vs CANDIDATE (capped) promotion check run in one shot",
        )
        ap.add_argument(
            "--hard-cap",
            action="store_true",
            help="enforce the cap as a HARD cap on --preset (chop continuously-active "
            "windows at the cap when no inactivity gap exists; W1 hard-cap fallback)",
        )
        ap.add_argument(
            "--vs-hard-cap",
            action="store_true",
            help="hard-cap fallback on --vs (default: same as --hard-cap)",
        )
        ap.add_argument(
            "--blob-fallback",
            action="store_true",
            help="R5 blob-fallback on --preset (after R4, force-chop any group still over
the "
            "cap at its midpoint + flag/log it; the continuously-active blob last resort)",
        )
        ap.add_argument(
            "--vs-blob-fallback",
            action="store_true",
            help="R5 blob-fallback on --vs (default: same as --blob-fallback)",
        )
        ap.add_argument(
            "--blob-factor",
            type=float,
            default=None,
            help="R5 magnitude gate (both presets): chop only groups > this × cap (default
4.0)",
        )
        ap.add_argument(
            "--inactivity-end",
```

```
            type=float,
            default=None,
            help="R4 rally-end inactivity trim (s) on --preset (0=off; trims post-rally
drift tail)",
        )
        ap.add_argument(
            "--vs-inactivity-end",
            type=float,
            default=None,
            help="R4 inactivity trim on --vs (default: same as --inactivity-end)",
        )
        ap.add_argument(
            "--inactivity-rally-thresh",
            type=float,
            default=None,
            help="R4 velocity above which motion counts as 'rally' (both presets; default
0.08)",
        )
        ap.add_argument(
            "--inactivity-min-density",
            type=float,
            default=None,
            help="R4 min trailing fast-fraction to stay 'in rally' (both presets; default
0.34)",
        )
        ap.add_argument(
            "--inactivity-temporal-density",
            action="store_true",
            help="R4 density over the trailing window's TEMPORAL length, not the active-
sample "
            "count, on --preset (finding-B fix; no-op under dense tracking, trims sparse-
track tails)",
        )
        ap.add_argument(
            "--vs-inactivity-temporal-density",
            action="store_true",
            help="R4 temporal-density on --vs (default: same as --inactivity-temporal-
density); use "
            "'--preset served --vs served --vs-inactivity-temporal-density' for the P2
promotion A/B",
        )
        ap.add_argument(
            "--tau-start",
            type=float,
            default=None,
            help="R2 tolerance-match start window (s); default 2.0 (forgives the service
window)",
        )
        ap.add_argument(
            "--tau-end",
            type=float,
            default=None,
            help="R2 tolerance-match end window (s); default 1.5 (keeps the rally-end
honest)",
        )
        ap.add_argument(
            "--guard-weight-merge",
            type=float,
            default=None,
            help="R1 over-seg guard: merge cost weight (default 2.0; merge hides a rally)",
        )
        ap.add_argument(
            "--guard-weight-split",
            type=float,
            default=None,
```

```
            help="R1 over-seg guard: split cost weight (default 1.0; split only fragments)",
        )
        ap.add_argument(
            "--guard-counts",
            action="store_true",
            help="R1 over-seg guard scores raw counts instead of normalized rates "
            "(rates are the default ? comparable across a structural mega?bounded change)",
        )
        ap.add_argument(
            "--min-duration",
            type=float,
            default=0.0,
            help="LONG-RALLY LENS (0=OFF): score ONLY rallies longer than this many seconds,
"
            "filtering BOTH predictions and golden GTs before matching. Strips local's "
            "short false-positive blips to reveal long-rally quality. Applies across the "
            "tIoU + R2 paths and to --vs (same filter on both sides).",
        )
        ap.add_argument(
            "--stratify",
            action="store_true",
            help="also print the long-rally stratified report: {all vs long(>--long-tau)} ×
"
            "{single vs multicourt} aggregates (court type from GOLDEN_VIDEOS.md / the "
            "manifest court_type field). The actual long-rally payoff table.",
        )
        ap.add_argument(
            "--long-tau",
            type=float,
            default=LONG_RALLY_TAU,
            help=f"the 'long' rally duration threshold (s) used by --stratify "
            f"(default {LONG_RALLY_TAU:g}; owner's North-star happy-path is >5s).",
        )
        ap.add_argument(
            "--json-out", default=None, help="optional path to write the full result JSON"
        )
        args = ap.parse_args()

        golden = load_golden(args.manifest, args.features_dir)
        base = _build_preset(args, "base")
        if args.vs:
            vs_mc = args.vs_min_crossings or args.min_crossings
            cand = _build_preset(args, "cand")
            result = compare(
                golden,
                base,
                cand,
                args.iou,
                base_min_crossings=args.min_crossings,
                cand_min_crossings=vs_mc,
                tau_start=args.tau_start,
                tau_end=args.tau_end,
                guard_weight_merge=args.guard_weight_merge,
                guard_weight_split=args.guard_weight_split,
                guard_use_rates=not args.guard_counts,
                min_duration=args.min_duration,
            )
            print(format_report(result["base"]))
            print("\n" + format_report(result["cand"]))
            d = result["delta"]
            print(
                f"\n## ? ({args.vs} mc={vs_mc} ? {args.preset} mc={args.min_crossings}), "
                f"n_paired={result['n_paired']}"
            )
            for k, dd in d.items():
```

```
808                 st = dd["sign_test"]
809                 print(
810                     f"  ?{k}: {dd['mean_delta']:+.3f}
[{dd['ci_low']:+.3f},{dd['ci_high']:+.3f}] "
811                     f"CI?0={dd['ci_excludes_zero']} sign(W/L)={st['wins']}/{st['losses']}
p={st['p_value']:.3f}"
812                 )
813             print(_format_guard(result["over_seg_guard"]))
814         else:
815             result = evaluate(
816                 golden,
817                 base,
818                 args.iou,
819                 args.min_crossings,
820                 tau_start=args.tau_start,
821                 tau_end=args.tau_end,
822                 min_duration=args.min_duration,
823             )
824             print(format_report(result))
825         if args.stratify:
826             # The long-rally payoff table always reports the BASE preset (the served/current
config),
827             # split by duration × court type. Independent of --min-duration (it does its own
split).
828             print(
829                 format_stratified(
830                     stratified_report(
831                         golden,
832                         base,
833                         args.iou,
834                         args.min_crossings,
835                         tau_start=args.tau_start,
836                         tau_end=args.tau_end,
837                         long_tau=args.long_tau,
838                     )
839                 )
840             )
841         if args.json_out:
842             with open(args.json_out, "w", encoding="utf-8") as fh:
843                 json.dump(result, fh, indent=2)
844             print(f"\n[written {args.json_out}]")
845
846
847     if __name__ == "__main__":
848         main()
849
```

8. assistant (2026-07-06T20:53:03.014Z)

Now let me check the remaining pieces: the segmentation_metrics for f1@0.5/f1@0.25, and the nightly_baseline.json anchor.

9. user (2026-07-06T20:53:04.522Z)

```
1       """Segmentation-aware metrics (course-correction C4) ? make over-segmentation visible.
2
3       Temporal IoU (``metrics.score``) is nearly blind to *fragmentation*: a single rally
4       chopped into three predicted pieces, or two rallies merged across a dead-time gap, can
5       still post a moderate mean-IoU. The temporal-action-segmentation / -localization
6       literature grades those failure modes with **F1@k** (segmental F1 at several overlap
7       thresholds) and **mAP@tIoU** (ranked, confidence-aware average precision over a tIoU
8       sweep). Reporting these alongside temporal-IoU is what lets a reconciler's split/merge/
9       tighten win actually show up ? and stops an IoU-chasing rule from shredding rally
10      structure invisibly. See ``docs/ANALYZERS_AND_RECONCILERS.md`` §13/C4.
11
```

```
    This is **additive**: new functions over the existing ``metrics`` matcher; nothing here
    changes ``score``/``compare``/LOVO behaviour. Pure (stdlib only), deterministic, no I/O
?
    so it runs in the GPU-free / numpy-free lanes too.

    Conventions match ``metrics``: an ``Interval`` is a ``(start, end)`` tuple of seconds.
    A ``ScoredInterval`` is ``(interval, confidence)`` ? the confidence is whatever the
scorer
    emits per predicted rally (e.g. the classifier probability), used only to *rank* for AP.

    Note on the segmental *edit* score: the classic TAS edit/Levenshtein score operates on
the
    ordered sequence of segment *labels* and is degenerate for a single action class ?
rally/
    background segments strictly alternate, so the edit distance collapses to (twice) the
    segment-count difference, adding nothing over ``segment_count_ratio``. So we
intentionally
    omit it and instead make over-segmentation explicit with ``merge_split_report`` (a
bipartite
    overlap-graph breakdown into merge/split/matched/missed/spurious ? see below), alongside
    ``f1_at_overlaps`` (F1@k) + ``mean_average_precision`` (mAP@tIoU). Those, not a single-
class
    edit score, are the over-segmentation-sensitive metrics for this binary rally setting.
    """

    from __future__ import annotations

    from typing import Dict, List, Sequence, Tuple

    from backend.eval.metrics import Interval, interval_iou, score

    #: A predicted interval plus the confidence used to rank it for average precision.
    ScoredInterval = Tuple[Interval, float]

    #: Default overlap thresholds for F1@k ? the TAS convention F1@{10,25,50}.
    DEFAULT_OVERLAPS: Tuple[float, ...] = (0.1, 0.25, 0.5)
    #: Default tIoU sweep for mean average precision ? the action-detection convention.
    DEFAULT_TIOUS: Tuple[float, ...] = (0.3, 0.4, 0.5, 0.6, 0.7)


    def f1_at_overlaps(
        preds: List[Interval],
        gts: List[Interval],
        overlaps: Sequence[float] = DEFAULT_OVERLAPS,
    ) -> Dict[float, float]:
        """Segmental F1 at each overlap threshold (``{overlap: f1}``) ? the TAS F1@k.

        Reuses the canonical greedy one-to-one matcher, so a fragmented prediction (3 pieces
        for 1 rally) scores extra false positives and a merged prediction misses a rally ?
        both drag F1 down where temporal-IoU alone would not flinch.
        """
        return {ov: score(preds, gts, iou_threshold=ov).f1 for ov in overlaps}


    def average_precision(
        scored_preds: Sequence[ScoredInterval],
        gts: List[Interval],
        iou_threshold: float = 0.5,
    ) -> float:
        """Average precision at a single tIoU (all-points / VOC-2010 interpolation).

        Predictions are ranked by descending confidence; each is a true positive iff it
        claims a still-unmatched ground-truth rally at IoU >= ``iou_threshold`` (highest
        available IoU wins, one GT per prediction). Returns AP in ``[0, 1]``. ``0.0`` when
        there are no ground-truth rallies or no predictions reach the threshold.
```

```
        """
        n_gt = len(gts)
        if n_gt == 0 or not scored_preds:
            return 0.0

        order = sorted(
            range(len(scored_preds)), key=lambda i: (-scored_preds[i][1], i)
        )  # conf desc, stable by index
        matched_gt = set()
        precisions: List[float] = []
        recalls: List[float] = []
        cum_tp = 0
        cum_fp = 0
        for i in order:
            interval, _conf = scored_preds[i]
            best_iou, best_gi = 0.0, -1
            for gi, g in enumerate(gts):
                if gi in matched_gt:
                    continue
                iou = interval_iou(interval, g)
                if iou >= iou_threshold and iou > best_iou:
                    best_iou, best_gi = iou, gi
            if best_gi >= 0:
                matched_gt.add(best_gi)
                cum_tp += 1
            else:
                cum_fp += 1
            precisions.append(cum_tp / (cum_tp + cum_fp))
            recalls.append(cum_tp / n_gt)

        # Monotone precision envelope (take the max precision to the right), then integrate
        # over recall steps: AP = sum_k (recall_k - recall_{k-1}) * precision_env_k.
        for i in range(len(precisions) - 2, -1, -1):
            precisions[i] = max(precisions[i], precisions[i + 1])
        ap = 0.0
        prev_recall = 0.0
        for p, r in zip(precisions, recalls):
            if r > prev_recall:
                ap += p * (r - prev_recall)
                prev_recall = r
        return ap


    def mean_average_precision(
        scored_preds: Sequence[ScoredInterval],
        gts: List[Interval],
        iou_thresholds: Sequence[float] = DEFAULT_TIOUS,
    ) -> float:
        """Mean of ``average_precision`` over a tIoU sweep (the standard mAP@tIoU)."""
        tious = list(iou_thresholds)
        if not tious:
            return 0.0
        return sum(average_precision(scored_preds, gts, t) for t in tious) / len(tious)


    def segment_count_ratio(preds: List[Interval], gts: List[Interval]) -> float:
        """Predicted-to-GT segment-count ratio ? a crude fragmentation diagnostic.

        ``> 1`` hints over-segmentation (more predicted pieces than rallies), ``< 1`` hints
        merging/misses. Only meaningful with ground truth present; returns ``0.0`` if
``gts``
        is empty (use F1@k / mAP for the graded picture).
        """
        if not gts:
            return 0.0
```

```
        return len(preds) / len(gts)


    def _overlaps(a: Interval, b: Interval) -> bool:
        """True iff two intervals share any positive temporal extent."""
        return min(a[1], b[1]) - max(a[0], b[0]) > 0.0


    def merge_split_report(preds: List[Interval], gts: List[Interval]) -> Dict[str, float]:
        """Explicit over-/under-segmentation breakdown via the bipartite overlap graph.

        Where ``segment_count_ratio`` only compares *counts* and ``f1_at_overlaps`` blends
merge
        and split into one F1, this names the two failure modes directly ? the diagnostic
that
        surfaced our local-segmenter over-MERGE (one giant window swallowing many rallies).
Build
        the bipartite graph (edge = any positive temporal overlap between a pred and a gt),
take its
        connected components, and classify each by ``(#preds, #gts)``:

          - ``(1,1)`` matched   ``(1,0)`` spurious (pred, no gt)   ``(0,1)`` missed (gt, no
pred)
          - ``(1,>1)`` **merge** ? one predicted window covers ?2 rallies (over-MERGE; our
failure)
          - ``(>1,1)`` **split** ? one rally cut across ?2 predictions (over-segmentation)
          - ``(>1,>1)`` complex (tangled many-to-many)

        Returns counts plus ``merge_rate`` (fraction of GT rallies swallowed inside a merge
group)
        and ``split_rate`` (fraction of GT rallies cut by ?2 predictions) ? the two headline
        over-segmentation rates. Pure, deterministic, stdlib-only.
        """
        n, m = len(preds), len(gts)
        p_adj: List[List[int]] = [[] for _ in range(n)]
        g_adj: List[List[int]] = [[] for _ in range(m)]
        for i, p in enumerate(preds):
            for j, g in enumerate(gts):
                if _overlaps(p, g):
                    p_adj[i].append(j)
                    g_adj[j].append(i)

        seen_p: set = set()
        seen_g: set = set()
        comps: List[Tuple[set, set]] = []
        for start in range(n):
            if start in seen_p:
                continue
            ps: set = set()
            gs: set = set()
            stack: List[Tuple[str, int]] = [("p", start)]
            while stack:
                side, k = stack.pop()
                if side == "p":
                    if k in ps:
                        continue
                    ps.add(k)
                    seen_p.add(k)
                    stack.extend(("g", j) for j in p_adj[k] if j not in gs)
                else:
                    if k in gs:
                        continue
                    gs.add(k)
                    seen_g.add(k)
                    stack.extend(("p", i) for i in g_adj[k] if i not in ps)
```

```
            comps.append((ps, gs))
        comps.extend(
            (set(), {j}) for j in range(m) if j not in seen_g
        )  # gts with no overlapping pred

        counts = {
            "matched": 0,
            "merges": 0,
            "splits": 0,
            "spurious": 0,
            "missed": 0,
            "complex": 0,
        }
        gts_merged = gts_split = 0
        for ps, gs in comps:
            a, b = len(ps), len(gs)
            if (a, b) == (1, 1):
                counts["matched"] += 1
            elif (a, b) == (1, 0):
                counts["spurious"] += 1
            elif (a, b) == (0, 1):
                counts["missed"] += 1
            elif a == 1 and b > 1:
                counts["merges"] += 1
                gts_merged += b
            elif b == 1 and a > 1:
                counts["splits"] += 1
                gts_split += 1
            else:
                counts["complex"] += 1
        out: Dict[str, float] = {k: float(v) for k, v in counts.items()}
        out["merge_rate"] = (gts_merged / m) if m else 0.0
        out["split_rate"] = (gts_split / m) if m else 0.0
        return out


    def segmentation_report(
        scored_preds: Sequence[ScoredInterval],
        gts: List[Interval],
        overlaps: Sequence[float] = DEFAULT_OVERLAPS,
        iou_thresholds: Sequence[float] = DEFAULT_TIOUS,
    ) -> Dict[str, object]:
        """One-call bundle: F1@k + mAP@tIoU + per-tIoU AP + count ratio + merge/split
breakdown.

        ``scored_preds`` carry confidences for AP; F1@k + the merge/split breakdown use the
bare
        intervals. Returns a plain dict (JSON-friendly, string keys) so it can ride next to
a
        ``compare`` result or land in the experiment leaderboard.
        """
        preds = [iv for iv, _conf in scored_preds]
        return {
            "f1_at_overlaps": {
                f"{ov:g}": v for ov, v in f1_at_overlaps(preds, gts, overlaps).items()
            },
            "ap_at_tiou": {
                f"{t:g}": average_precision(scored_preds, gts, t) for t in iou_thresholds
            },
            "mean_ap": mean_average_precision(scored_preds, gts, iou_thresholds),
            "segment_count_ratio": segment_count_ratio(preds, gts),
            "merge_split": merge_split_report(preds, gts),
            "num_pred": len(preds),
            "num_gt": len(gts),
        }
```

254

10. user (2026-07-06T20:53:04.756Z)

eval_baselines\nightly_baseline.json

11. assistant (2026-07-06T20:53:12.942Z)

I found the `f1_at_overlaps` scorer. Note that `f1@0.5`/`f1@0.25` come from `segmentation_metrics.f1_at_overlaps` (which uses `metrics.score` for the greedy IoU-matched F1 at that overlap threshold), NOT from `rally_seg_eval` directly. Now let me check the nightly_baseline.json anchor and the `metrics.score` signature (to confirm the greedy tIoU-matcher underlying f1@k).

12. user (2026-07-06T20:53:14.445Z)


```
1       {
2         "_doc": "Frozen nightly rally-quality regression baseline. Regenerate with `python
scripts/nightly_regression.py --update-baseline` after an intended improvement or a corpus
change. Tied to the LOCAL golden corpus (not committed).",
3         "configs": {
4           "baseline_off": {
5             "aggregate": {
6               "dend_abs_median": 41.77921047714383,
7               "dstart_abs_median": 42.467243955066174,
8               "f1@0.25": 0.39981006941991765,
9               "f1@0.5": 0.2366145803119009,
10              "merge_rate": 0.6380248744753217,
11              "precision": 0.2577557018251538,
12              "recall": 0.22719343607869802,
13              "segment_count_ratio": 0.5824947574930804,
14              "split_rate": 0.0,
15              "tol_f1": 0.1382064040388172,
16              "tol_precision": 0.14536686157033415,
17              "tol_recall": 0.13445461315285676
18            },
19            "iou": 0.5,
20            "min_crossings": 0,
21            "n": 15,
22            "note": "Pre-R1-flip all-OFF windowing \u2014 the historical floor; pins the value
the milestone buys.",
23            "per_video": {
24              "Badminton_BXH_2": {
25                "f1@0.25": 0.0,
26                "f1@0.5": 0.0,
27                "merge_count": 3,
28                "merge_rate": 1.0,
29                "num_gt": 44,
30                "num_pred": 4,
31                "split_count": 0,
32                "tol_f1": 0.0
33              },
34              "Boxhill_Doubles": {
35                "f1@0.25": 0.10526315789473685,
36                "f1@0.5": 0.03508771929824561,
37                "merge_count": 7,
38                "merge_rate": 0.9534883720930233,
39                "num_gt": 43,
40                "num_pred": 14,
41                "split_count": 0,
42                "tol_f1": 0.0
43              },
44              "GX010128": {
45                "f1@0.25": 0.14925373134328357,
```

```
46                "f1@0.5": 0.05970149253731343,
47                "merge_count": 12,
48                "merge_rate": 0.9615384615384616,
49                "num_gt": 52,
50                "num_pred": 15,
51                "split_count": 0,
52                "tol_f1": 0.0
53              },
54              "GX010137": {
55                "f1@0.25": 0.9253731343283583,
56                "f1@0.5": 0.5970149253731343,
57                "merge_count": 3,
58                "merge_rate": 0.17647058823529413,
59                "num_gt": 34,
60                "num_pred": 33,
61                "split_count": 0,
62                "tol_f1": 0.4477611940298507
63              },
64              "GX010141": {
65                "f1@0.25": 0.5416666666666666,
66                "f1@0.5": 0.3333333333333333,
67                "merge_count": 5,
68                "merge_rate": 0.6551724137931034,
69                "num_gt": 29,
70                "num_pred": 19,
71                "split_count": 0,
72                "tol_f1": 0.3333333333333333
73              },
74              "GX020094": {
75                "f1@0.25": 0.7333333333333334,
76                "f1@0.5": 0.4444444444444445,
77                "merge_count": 3,
78                "merge_rate": 0.15,
79                "num_gt": 40,
80                "num_pred": 50,
```

13. user (2026-07-06T20:53:15.791Z)

```
134:        "mahadevpura_1": {
320:        "mahadevpura_1": {
506:        "mahadevpura_1": {
```

14. assistant (2026-07-06T20:53:22.191Z)

There are three config presets. Let me look at the config names and the mahadevpura_1 entries, plus identify which is `default`/`milestone`.

15. user (2026-07-06T20:53:24.844Z)

No matches found

16. user (2026-07-06T20:53:25.574Z)

```
125               "f1@0.25": 0.8118811881188118,
126               "f1@0.5": 0.5346534653465348,
127               "merge_count": 4,
128               "merge_rate": 0.16326530612244897,
129               "num_gt": 49,
130               "num_pred": 52,
131               "split_count": 0,
132               "tol_f1": 0.297029702970297
133             },
134             "mahadevpura_1": {
135               "f1@0.25": 0.0,
```

```
136               "f1@0.5": 0.0,
137               "merge_count": 1,
138               "merge_rate": 1.0,
139               "num_gt": 10,
140               "num_pred": 1,
141               "split_count": 0,
142               "tol_f1": 0.0
143             },
144             "mahadevpura_2": {
145               "f1@0.25": 0.0,
146               "f1@0.5": 0.0,
147               "merge_count": 3,
148               "merge_rate": 0.975,
149               "num_gt": 40,
150               "num_pred": 5,
151               "split_count": 0,
152               "tol_f1": 0.0
153             },
154             "mahadevpura_singles": {
```

17. assistant (2026-07-06T20:53:32.920Z)

```
The first mahadevpura_1 (line 134) is under `baseline_off`. Let me find the config names for all
three blocks and read the mahadevpura_1 entries at lines 320 and 506.
```

18. user (2026-07-06T20:53:34.587Z)

```
No matches found
```

19. user (2026-07-06T20:53:35.930Z)

```
318               "tol_f1": 0.5405405405405406
319             },
320             "mahadevpura_1": {
321               "f1@0.25": 0.0,
322               "f1@0.5": 0.0,
323               "merge_count": 2,
324               "merge_rate": 0.8,
325               "num_gt": 10,
326               "num_pred": 2,
327               "split_count": 0,
328               "tol_f1": 0.0
329             },
330             "mahadevpura_2": {
331               "f1@0.25": 0.525,
```

20. user (2026-07-06T20:53:36.546Z)

```
504               "tol_f1": 0.5405405405405406
505             },
506             "mahadevpura_1": {
507               "f1@0.25": 0.0,
508               "f1@0.5": 0.0,
509               "merge_count": 2,
510               "merge_rate": 0.8,
511               "num_gt": 10,
512               "num_pred": 2,
513               "split_count": 0,
514               "tol_f1": 0.0
515             },
516             "mahadevpura_2": {
517               "f1@0.25": 0.525,
```

21. assistant (2026-07-06T20:53:38.878Z)

Let me find the top-level config keys (the grep pattern needs adjusting for JSON indentation).

22. user (2026-07-06T20:53:40.537Z)

```
4:    "baseline_off": {
190:    "default": {
376:    "milestone": {
568:    "metric_tol": 0.005,
569:    "over_seg_tol": 0,
570:    "rate_tol": 0.01
```

23. user (2026-07-06T20:53:41.532Z)

```
22:      "note": "Pre-R1-flip all-OFF windowing \u2014 the historical floor; pins the value the
milestone buys.",
208:      "note": "SERVED production windowing \u2014 now the R1 milestone (cap=6 + R4) after
the #204 flip.",
394:      "note": "R3 cap=6 s + R4 rally-END inactivity trim \u2014 the shipped milestone (==
default post-#204).",
```

24. assistant (2026-07-06T20:53:50.202Z)

Now I have everything. The three config blocks are `baseline_off` (line 4), `default` (line 190), and `milestone` (line 376). The `default` preset's `per_video.mahadevpura_1` is at line 320, and `milestone` at line 506 ? both identical:

- `mahadevpura_1`: `f1@0.25=0.0`, `f1@0.5=0.0`, `tol_f1=0.0`, `merge_count=2`, `merge_rate=0.8`, `num_gt=10`, `num_pred=2`, `split_count=0`.

Let me verify the `metrics.score` signature (underlying f1@k) and `windowing.PRESETS`/`SERVED` to confirm the served/default preset's windowing params, so my BASELINE config choice is accurate and I can cite what the served preset actually uses.

25. user (2026-07-06T20:53:52.382Z)

```
32:@dataclass
41:@dataclass
91:@dataclass
101:    precision: float
102:    recall: float
103:    f1: float
112:def score(
133:    precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
134:    recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
135:    f1 = (
```

26. user (2026-07-06T20:53:54.008Z)

```
5:on a *recall-oriented PROPOSAL* windowing (``vt=0.005, merge_gap=1.0, min_window=0.5``):
6:deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED pipeline
7:(``TrajectoryHybridSegmenter.process_video``) windows COARSELY (``vt=0.02, merge_gap=2.0,
14:SERVED preset is **single-sourced from the same Pydantic config defaults production reads**
20:The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED
39:    "SERVED",
50:    """A named (velocity_thresh, merge_gap, min_window_duration) windowing operating point.
52:    ``velocity_thresh`` ? normalised per-second shuttle velocity above which a frame counts
as
53:    "in flight"; ``merge_gap`` ? seconds within which active frames coalesce into one window;
54:    ``min_window_duration`` ? windows shorter than this are dropped. These are exactly the
three
59:    velocity_thresh: float
60:    merge_gap: float
61:    min_window_duration: float
64:    max_window_duration: float = 0.0
```

```
65:    #: When True, ``max_window_duration`` is a HARD cap ? an over-long window with no
internal
76:    #: tracking inflates). No-op under dense tracking; gated default-OFF until measured on
SERVED.
79:    #: longer than ``blob_factor`` × ``max_window_duration`` (the gap-less, R4-survived blob
?
86:        """``(velocity_thresh, merge_gap, min_window_duration)`` ? the legacy tuple shape the
88:        bounded-windowing knob ``max_window_duration`` is passed separately by
``build_windows``.)"""
89:        return (self.velocity_thresh, self.merge_gap, self.min_window_duration)
94:            "velocity_thresh": self.velocity_thresh,
95:            "merge_gap": self.merge_gap,
96:            "min_window_duration": self.min_window_duration,
97:            "max_window_duration": self.max_window_duration,
109:    """The SERVED windowing, lifted from the SAME Pydantic config defaults production reads.
112:      - velocity:   ``idx_cfg[<family>].velocity_threshold`` (default 0.02 ?
wasb/fusion/tracknet)
113:      - merge_gap:  ``idx_cfg.dead_time_merge_gap`` (default 2.0)
117:    point: change a served default in ``config/models.py`` and the eval SERVED preset
follows,
122:        name="served",
123:        velocity_thresh=float(
124:            idx.wasb.velocity_threshold
126:        merge_gap=float(idx.dead_time_merge_gap),
127:        min_window_duration=float(idx.min_action_window_duration),
128:        max_window_duration=float(
129:            idx.max_window_duration
147:SERVED: WindowingPreset = _served_preset_from_defaults()
151:#: win here must be re-confirmed on SERVED before promotion. Mirrors the historical
154:    name="proposal", velocity_thresh=0.005, merge_gap=1.0, min_window_duration=0.5
158:PRESETS: Dict[str, WindowingPreset] = {SERVED.name: SERVED, PROPOSAL.name: PROPOSAL}
162:    """The SERVED windowing a *specific* live config serves (overrides honoured).
167:    default in ``config.json`` ? the module-level :data:`SERVED` is the *default* operating
point;
173:        name=f"served:{family}",
174:        velocity_thresh=float(
175:            family_cfg.get("velocity_threshold", SERVED.velocity_thresh)
177:        merge_gap=float(idx.get("dead_time_merge_gap", SERVED.merge_gap)),
178:        min_window_duration=float(
179:            idx.get("min_action_window_duration", SERVED.min_window_duration)
181:        max_window_duration=float(
182:            idx.get("max_window_duration", SERVED.max_window_duration)
184:        hard_cap=bool(idx.get("window_hard_cap", SERVED.hard_cap)),
185:        inactivity_end=float(idx.get("window_inactivity_end", SERVED.inactivity_end)),
187:            idx.get("inactivity_rally_thresh", SERVED.inactivity_rally_thresh)
190:            idx.get("inactivity_min_density", SERVED.inactivity_min_density)
193:            idx.get("inactivity_temporal_density", SERVED.inactivity_temporal_density)
195:        blob_fallback=bool(idx.get("window_blob_fallback", SERVED.blob_fallback)),
196:        blob_factor=float(idx.get("window_blob_factor", SERVED.blob_factor)),
229:        velocity_thresh=vt,
230:        merge_gap=mg,
231:        min_window_duration=mwd,
232:        max_window_duration=preset.max_window_duration,
```

27. assistant (2026-07-06T20:54:00.966Z)

I have everything I need. Key findings for the windowing config:

- **`calibrate_wasb.py` BASELINE** = `(0.02, 2.0, 2.0)` = `(velocity_thresh, merge_gap, min_window_duration)` ? these are exactly the `trajectory_to_action_windows` defaults, and the task explicitly asks to "copy the exact call pattern + the BASELINE config `(0.02, 2.0, 2.0)`".
- The nightly `default`/`milestone` presets additionally use `max_window_duration=6` + R4 inactivity-end trim (the SERVED production config, cap=6+R4), which is why their `mahadevpura_1` differs slightly from `baseline_off`. But the task is explicit: use the calibrate_wasb BASELINE `(0.02, 2.0, 2.0)` windowing exactly. I'll use that as the primary path and note the divergence.

Now let me check `metrics.score` full signature to confirm the `iou_threshold` kwarg (used by `f1_at_overlaps`).

28. user (2026-07-06T20:54:02.752Z)

```
91      @dataclass
92      class Score:
93          """Aggregate metrics for one (predictions vs ground-truth) evaluation."""
94
95          iou_threshold: float
96          num_pred: int
97          num_gt: int
98          true_positives: int
99          false_positives: int
100         false_negatives: int
101         precision: float
102         recall: float
103         f1: float
104         mean_iou: float  # over matched pairs (0.0 if none)
105         mean_start_error: float  # mean |pred_start - gt_start| over matches, seconds
106         mean_end_error: float  # mean |pred_end - gt_end| over matches, seconds
107
108         def as_dict(self) -> dict:
109             return self.__dict__.copy()
110
111
112     def score(
113         preds: List[Interval],
114         gts: List[Interval],
115         iou_threshold: float = 0.5,
116         match_result: Optional[MatchResult] = None,
117     ) -> Score:
118         """Compute precision/recall/F1, mean matched IoU, and boundary errors.
119
120         Pass a precomputed `match_result` to avoid re-matching (e.g. when also
121         extracting the failure lists); otherwise matching is done here.
122         """
123         mr = (
124             match_result
125             if match_result is not None
126             else match_intervals(preds, gts, iou_threshold)
127         )
128
129         tp = len(mr.matches)
130         fp = len(mr.unmatched_pred_indices)
131         fn = len(mr.unmatched_gt_indices)
132
133         precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
134         recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
135         f1 = (
136             (2 * precision * recall / (precision + recall))
137             if (precision + recall) > 0
138             else 0.0
139         )
140
```

29. assistant (2026-07-06T20:55:30.340Z)

I now have all the interfaces confirmed. I have everything needed to write the script. Here is the complete `score_local.py`.

```python
#!/usr/bin/env python3
"""score_local.py ? score WASB trajectory CSVs against golden rally labels using THIS
repo's own windowing + metrics, so a torch-2.x GPU-resident-decode trajectory can be
```

```
compared against the CPU trajectory and the cached baseline trajectory for regression.

Run from the repo root:
    C:\\Users\\avidu\\Projects\\khelsutra-guru\\badminton-highlight-indexer

For EACH trajectory CSV (baseline / cpu / gpu):
    TrackNetRunner.parse_trajectory_csv(path)                  -> List[TrajectoryPoint]
    TrackNetRunner.trajectory_to_action_windows(points, ...)   -> List[dict] windows
    [(w["start"], w["end"]) for w in windows]                  -> predicted intervals
Golden GT via:
    load_gt_intervals(labels_csv, strict=False)                -> List[(start,end)]

Scored with BOTH:
    - tIoU F1@0.5 and F1@0.25 : segmentation_metrics.f1_at_overlaps (greedy 1:1 tIoU match
      via metrics.score(iou_threshold=...)) ? the same greedy tIoU-gated F1 rally_seg_eval
      reports as f1@0.5 / f1@0.25.
    - tol_f1 / tol_precision / tol_recall : tolerance_metrics.match_1to1 (strict 1:1
      tolerance-match, TAU_START=2.0 / TAU_END=1.5).

Windowing config is the calibrate_wasb.py BASELINE = (velocity_thresh, merge_gap,
min_window_duration) = (0.02, 2.0, 2.0), invoked exactly as calibrate_wasb.make_predict_fn
does. Everything runs offline on CPU; no torch / TF / GPU is imported here.

The KEY question this answers: is gpu ~= cpu ~= baseline (no regression)?
"""

from __future__ import annotations

import argparse
import os
import sys
import traceback
from typing import Any, Dict, List, Optional, Tuple

Interval = Tuple[float, float]
```

---- calibrate_wasb.py BASELINE config: (velocity_thresh, merge_gap, min_window_d

These are the trajectory_to_action_windows defaults; calibrate_wasb.BASELINE == (0

```
BASELINE_VELOCITY_THRESH = 0.02
BASELINE_MERGE_GAP = 2.0
BASELINE_MIN_WINDOW_DURATION = 2.0
```

eval_baselines/nightly_baseline.json anchor for mahadevpura_1 (configs.default.per_

configs.milestone.per_video ? identical). Printed as a sanity anchor next to our table.

```
ANCHOR_DEFAULT_MAHADEVPURA_1 = {
    "f1@0.5": 0.0,
    "f1@0.25": 0.0,
    "tol_f1": 0.0,
    "merge_count": 2,
    "merge_rate": 0.8,
    "num_gt": 10,
    "num_pred": 2,
    "split_count": 0,
    "note": "configs.default (== configs.milestone) ? SERVED cap=6+R4 windowing, "
    "NOT the (0.02,2.0,2.0) baseline this script uses; expect small differences.",
}
```

The all-OFF historical floor (configs.baseline_off) is the closest match to this script's (0.02, 2.0, 2.0) windowing (no cap / no R4), so it is the more apples-to-apples anchor.

```
ANCHOR_BASELINE_OFF_MAHADEVPURA_1 = {
    "f1@0.5": 0.0,
    "f1@0.25": 0.0,
```

```
    "tol_f1": 0.0,
    "merge_count": 1,
    "merge_rate": 1.0,
    "num_gt": 10,
    "num_pred": 1,
    "split_count": 0,
    "note": "configs.baseline_off ? all-OFF (no cap / no R4) windowing, closest to the "
    "(0.02,2.0,2.0) config used here.",
}


def _import_repo() -> Dict[str, Any]:
    """Import the repo interfaces, adapting if a name/signature differs from expectations.

    The repo is the source of truth: if any import fails we surface a clear error rather
    than silently mis-scoring.
    """
    mods: Dict[str, Any] = {}
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

    mods["TrackNetRunner"] = TrackNetRunner

    from backend.eval.gt_loader import load_gt_intervals

    mods["load_gt_intervals"] = load_gt_intervals

    from backend.eval import tolerance_metrics as tm

    mods["tm"] = tm

    # tIoU F1@k: segmentation_metrics.f1_at_overlaps is the greedy 1:1 tIoU-matched F1 at
    # each overlap threshold (built on metrics.score) ? exactly rally_seg_eval's f1@0.25/f1@0.5.
    try:
        from backend.eval import segmentation_metrics as sm

        mods["sm"] = sm
    except Exception:  # pragma: no cover - fallback only if the module is unavailable
        mods["sm"] = None

    # Fallback greedy tIoU-F1 matcher (metrics.score) in case f1_at_overlaps is missing.
    try:
        from backend.eval import metrics as met

        mods["metrics"] = met
    except Exception:  # pragma: no cover
        mods["metrics"] = None

    return mods


def parse_and_window(
    csv_path: str,
    TrackNetRunner: Any,
    fps: float,
    frame_width: float,
) -> List[Interval]:
    """csv -> points -> BASELINE action windows -> predicted rally intervals.

    Mirrors calibrate_wasb.make_predict_fn's exact call pattern with the BASELINE config.
    """
    points = TrackNetRunner.parse_trajectory_csv(csv_path)
    wins = TrackNetRunner.trajectory_to_action_windows(
        points,
        fps=fps,
        frame_width=frame_width,
```

```
        velocity_thresh=BASELINE_VELOCITY_THRESH,
        merge_gap=BASELINE_MERGE_GAP,
        min_window_duration=BASELINE_MIN_WINDOW_DURATION,
    )
    return [(float(w["start"]), float(w["end"])) for w in wins]


def tiou_f1(preds: List[Interval], gts: List[Interval], iou: float, mods: Dict[str, Any]) ->
float:
    """Greedy 1:1 temporal-IoU-gated F1 at overlap `iou` (rally_seg_eval's f1@k)."""
    sm = mods.get("sm")
    if sm is not None:
        return float(sm.f1_at_overlaps(preds, gts, overlaps=(iou,))[iou])
    # Replicated fallback: the same greedy tIoU-matched F1 via metrics.score.
    met = mods.get("metrics")
    if met is not None:
        return float(met.score(preds, gts, iou_threshold=iou).f1)
    return _greedy_tiou_f1(preds, gts, iou)


def _greedy_tiou_f1(preds: List[Interval], gts: List[Interval], iou_thresh: float) -> float:
    """Last-resort self-contained greedy tIoU-F1 (match if temporal-IoU >= thresh, 1:1)."""
    if not preds or not gts:
        return 0.0

    def _iou(a: Interval, b: Interval) -> float:
        inter = max(0.0, min(a[1], b[1]) - max(a[0], b[0]))
        union = (a[1] - a[0]) + (b[1] - b[0]) - inter
        return inter / union if union > 0 else 0.0

    pairs = []
    for i, p in enumerate(preds):
        for j, g in enumerate(gts):
            v = _iou(p, g)
            if v >= iou_thresh:
                pairs.append((-v, i, j))
    pairs.sort()
    used_p: set = set()
    used_g: set = set()
    tp = 0
    for _, i, j in pairs:
        if i in used_p or j in used_g:
            continue
        used_p.add(i)
        used_g.add(j)
        tp += 1
    fp = len(preds) - tp
    fn = len(gts) - tp
    precision = tp / (tp + fp) if (tp + fp) else 0.0
    recall = tp / (tp + fn) if (tp + fn) else 0.0
    return (2 * precision * recall / (precision + recall)) if (precision + recall) else 0.0


def tolerance_scores(
    preds: List[Interval], gts: List[Interval], mods: Dict[str, Any]
) -> Tuple[float, float, float]:
    """tol_f1 / tol_precision / tol_recall via tolerance_metrics.match_1to1
(TAU_START/TAU_END)."""
    tm = mods["tm"]
    # match_1to1 -> (matches, precision, recall, f1); uses TAU_START=2.0 / TAU_END=1.5 by
default.
    _matches, p, r, f1 = tm.match_1to1(preds, gts, tm.TAU_START, tm.TAU_END)
    return float(f1), float(p), float(r)
```

```
def score_one(
    label: str,
    csv_path: str,
    gts: List[Interval],
    mods: Dict[str, Any],
    fps: float,
    frame_width: float,
) -> Dict[str, Any]:
    """Full metric row for one trajectory CSV. Wrapped in try/except by the caller."""
    preds = parse_and_window(csv_path, mods["TrackNetRunner"], fps, frame_width)
    f1_25 = tiou_f1(preds, gts, 0.25, mods)
    f1_50 = tiou_f1(preds, gts, 0.5, mods)
    tol_f1, tol_p, tol_r = tolerance_scores(preds, gts, mods)
    return {
        "label": label,
        "ok": True,
        "n_pred": len(preds),
        "n_gt": len(gts),
        "f1@0.25": f1_25,
        "f1@0.5": f1_50,
        "tol_f1": tol_f1,
        "tol_precision": tol_p,
        "tol_recall": tol_r,
    }


def _fmt(v: Any) -> str:
    if isinstance(v, float):
        return f"{v:.3f}"
    return str(v)


def print_table(rows: List[Dict[str, Any]], n_gt: int, name: str, tm: Any) -> None:
    tau_s = getattr(tm, "TAU_START", 2.0)
    tau_e = getattr(tm, "TAU_END", 1.5)
    print()
    print("=" * 78)
    print(f"score_local ? {name}  (windowing BASELINE vt/mg/mwd = "
          f"{BASELINE_VELOCITY_THRESH}/{BASELINE_MERGE_GAP}/{BASELINE_MIN_WINDOW_DURATION}; "
          f"tol ?_start/?_end = {tau_s}/{tau_e})")
    print(f"golden rallies (n_gt): {n_gt}")
    print("=" * 78)

    header = ["traj", "n_pred", "f1@0.25", "f1@0.5", "tol_f1", "tol_P", "tol_R"]
    widths = [10, 7, 8, 8, 8, 7, 7]

    def _row(cells: List[str]) -> str:
        return "  ".join(str(c).ljust(w) for c, w in zip(cells, widths))

    print(_row(header))
    print(_row(["-" * w for w in widths]))
    for r in rows:
        if r.get("ok"):
            print(_row([
                r["label"],
                str(r["n_pred"]),
                _fmt(r["f1@0.25"]),
                _fmt(r["f1@0.5"]),
                _fmt(r["tol_f1"]),
                _fmt(r["tol_precision"]),
                _fmt(r["tol_recall"]),
            ]))
        else:
            print(_row([r["label"], "ERR", "-", "-", "-", "-", "-"]))
            print(f"    ! {r['label']} failed: {r.get('error', 'unknown error')}")
```

```
    # eval_baselines anchor for mahadevpura_1, printed next to the table.
    print()
    print("-" * 78)
    print("eval_baselines/nightly_baseline.json anchor ? mahadevpura_1:")
    a = ANCHOR_BASELINE_OFF_MAHADEVPURA_1
    print(f"  [baseline_off] f1@0.25={a['f1@0.25']}  f1@0.5={a['f1@0.5']}  "
          f"tol_f1={a['tol_f1']}  n_pred={a['num_pred']}  n_gt={a['num_gt']}  "
          f"merge_count={a['merge_count']}  ({a['note']})")
    d = ANCHOR_DEFAULT_MAHADEVPURA_1
    print(f"  [default/milestone] f1@0.25={d['f1@0.25']}  f1@0.5={d['f1@0.5']}  "
          f"tol_f1={d['tol_f1']}  n_pred={d['num_pred']}  n_gt={d['num_gt']}  "
          f"merge_count={d['merge_count']}  ({d['note']})")

    # Regression verdict: gpu ~= cpu ~= baseline?
    print()
    print("-" * 78)
    ok = {r["label"]: r for r in rows if r.get("ok")}
    if {"baseline", "cpu", "gpu"} <= set(ok):
        keys = ("n_pred", "f1@0.25", "f1@0.5", "tol_f1")

        def _close(a_row: Dict[str, Any], b_row: Dict[str, Any]) -> bool:
            for k in keys:
                av, bv = a_row[k], b_row[k]
                if isinstance(av, int) and isinstance(bv, int):
                    if av != bv:
                        return False
                else:
                    if abs(float(av) - float(bv)) > 1e-6:
                        return False
            return True

        gpu_eq_cpu = _close(ok["gpu"], ok["cpu"])
        gpu_eq_base = _close(ok["gpu"], ok["baseline"])
        cpu_eq_base = _close(ok["cpu"], ok["baseline"])
        verdict = "PASS (gpu == cpu == baseline)" if (gpu_eq_cpu and gpu_eq_base and
cpu_eq_base) else \
                  "REVIEW ? trajectories diverge"
        print(f"REGRESSION CHECK: {verdict}")
        print(f"  gpu==cpu: {gpu_eq_cpu}   gpu==baseline: {gpu_eq_base}   cpu==baseline:
{cpu_eq_base}")
        if not (gpu_eq_cpu and gpu_eq_base and cpu_eq_base):
            print("  (differences in n_pred / f1 / tol_f1 mean the GPU decode path produced a "
                  "different trajectory ? investigate before trusting the GPU path.)")
    else:
        missing = {"baseline", "cpu", "gpu"} - set(ok)
        print(f"REGRESSION CHECK: skipped ? missing/failed trajectories: {sorted(missing)}")
    print("=" * 78)


def main(argv: Optional[List[str]] = None) -> int:
    ap = argparse.ArgumentParser(
        description="Score WASB trajectory CSVs vs golden rallies with this repo's windowing +
metrics."
    )
    ap.add_argument("--dir", default=".", help="directory holding the CSVs (default: repo root /
cwd)")
    ap.add_argument("--cpu-traj", default="cpu_traj.csv")
    ap.add_argument("--gpu-traj", default="gpu_traj.csv")
    ap.add_argument("--baseline-traj", default="baseline_traj.csv")
    ap.add_argument("--labels", default="g1_labels.csv", help="golden .rallies.csv labels")
    ap.add_argument("--fps", type=float, default=59.94)
    ap.add_argument("--frame-width", type=float, default=1920.0)
    ap.add_argument("--name", default="mahadevpura_1")
    args = ap.parse_args(argv)
```

```
    try:
        mods = _import_repo()
    except Exception as e:  # import problem ? cannot proceed at all
        print("FATAL: could not import repo interfaces. Run from the repo root "
              "(C:\\Users\\avidu\\Projects\\khelsutra-guru\\badminton-highlight-indexer).",
              file=sys.stderr)
        traceback.print_exc()
        return 2

    def _p(fname: str) -> str:
        return fname if os.path.isabs(fname) else os.path.join(args.dir, fname)

    labels_path = _p(args.labels)

    # Load GT once (strict=False = calibrate_wasb's lenient semantics: skip bad rows, but log).
    try:
        gts = mods["load_gt_intervals"](labels_path, strict=False)
    except Exception as e:
        print(f"FATAL: failed to load golden labels {labels_path}: {e}", file=sys.stderr)
        traceback.print_exc()
        return 2
    if not gts:
        print(f"FATAL: no usable golden rallies in {labels_path}", file=sys.stderr)
        return 2

    trajectories = [
        ("baseline", _p(args.baseline_traj)),
        ("cpu", _p(args.cpu_traj)),
        ("gpu", _p(args.gpu_traj)),
    ]

    rows: List[Dict[str, Any]] = []
    for label, path in trajectories:
        if not os.path.exists(path):
            rows.append({"label": label, "ok": False, "error": f"file not found: {path}"})
            continue
        try:
            rows.append(score_one(label, path, gts, mods, args.fps, args.frame_width))
        except Exception as e:  # one bad trajectory must not kill the table
            rows.append({"label": label, "ok": False, "error": f"{type(e).__name__}: {e}"})
            traceback.print_exc()

    print_table(rows, len(gts), args.name, mods["tm"])
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Notes

Exact functions / signatures used (all read from the repo ? source of truth):

- `backend/pipeline/detectors/tracknet_runner.py`
  - `TrackNetRunner.parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]` (staticmethod; reads `Frame,Visibility,X,Y` CSV).
  - `TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, chunk_start=0.0, velocity_thresh=0.02, merge_gap=2.0, min_window_duration=2.0, ...)` (staticmethod). Returns a `List[Dict[str, Any]]`; each window dict has `"start"` and `"end"` in **seconds** (plus `active_frame_count`, `peak_velocity`, etc.). I call it exactly as `calibrate_wasb.make_predict_fn` does ? passing only `fps`, `frame_width`, `velocity_thresh`, `merge_gap`, `min_window_duration` ? and take `[(w["start"], w["end"]) for w in wins]`.

- `backend/eval/calibrate_wasb.py`
  - `BASELINE: Config = (0.02, 2.0, 2.0)` = `(velocity_thresh, merge_gap, min_window_duration)` ? copied verbatim as the windowing config. Its `make_predict_fn` is the call pattern I replicated.

- `backend/eval/tolerance_metrics.py`
  - `match_1to1(preds, gts, tau_start=TAU_START, tau_end=TAU_END) -> (matches, precision, recall, f1)` ? strict 1:1 tolerance-match (Hungarian if scipy present, else deterministic greedy). Used for `tol_f1/tol_precision/tol_recall`.
  - `TAU_START = 2.0`, `TAU_END = 1.5` ? used as-is (matches your spec of 2.0/1.5).

- `backend/eval/rally_seg_eval.py` / `backend/eval/segmentation_metrics.py`
  - The tIoU-gated `f1@0.5`/`f1@0.25` that `rally_seg_eval` reports come from `segmentation_metrics.f1_at_overlaps(preds, gts, overlaps=(...))`, which is `{ov: metrics.score(preds, gts, iou_threshold=ov).f1}` ? i.e. the canonical **greedy 1:1 temporal-IoU-matched F1** at that overlap threshold. I call `f1_at_overlaps` directly (primary path), fall back to `metrics.score(iou_threshold=iou).f1` if that module is unavailable, and include a self-contained greedy tIoU-F1 as a last resort so the table never dies on an import quirk.

- `backend/eval/gt_loader.py`
  - `load_gt_intervals(csv_path, *, strict=False) -> List[(start, end)]` ? used with `strict=False` (calibrate_wasb's lenient "skip bad rows, log skips" semantics). Accepts both `start/end` and `start_time/end_time` conventions.

Windowing config used: the calibrate_wasb `BASELINE` = `velocity_thresh=0.02, merge_gap=2.0, min_window_duration=2.0` (these are also the `trajectory_to_action_windows` defaults). No cap / R4 / blob-fallback ? matching `configs.baseline_off` in the nightly baseline. All three trajectories go through the identical windowing so any table difference is purely the trajectory (i.e. the GPU-decode path), which is the regression signal you want.

eval_baselines anchor found for `mahadevpura_1`: there are three presets in `eval_baselines/nightly_baseline.json`: `baseline_off` (line 4), `default` (line 190), `milestone` (line 376).
- `configs.default.per_video.mahadevpura_1` **and** `configs.milestone.per_video.mahadevpura_1` are **identical**: `f1@0.5 = 0.0`, `f1@0.25 = 0.0`, `tol_f1 = 0.0`, `num_pred = 2`, `num_gt = 10`, `merge_count = 2`, `merge_rate = 0.8`, `split_count = 0`. (Both use the SERVED cap=6+R4 windowing, per their `note` fields.)
- `configs.baseline_off.per_video.mahadevpura_1`: `f1@0.5 = 0.0`, `f1@0.25 = 0.0`, `tol_f1 = 0.0`, `num_pred = 1`, `num_gt = 10`, `merge_count = 1`, `merge_rate = 1.0`.

The script prints **both** anchors, because the `default`/`milestone` numbers you asked for use a *different* windowing (cap=6 + R4) than this script's `(0.02, 2.0, 2.0)`; `baseline_off` is the more apples-to-apples anchor for the config this script actually runs. All three golden anchors for `mahadevpura_1` are 0.0 on f1/tol_f1 (this clip's mega-window is a known over-merge failure ? `num_gt=10` rallies collapse into 1-2 predicted windows), so treat the anchor as a "trajectories parse and window sanely" sanity check, not a quality target.

Robustness: each trajectory is wrapped in its own try/except (a missing/broken CSV yields an `ERR` row, not a crashed table); imports adapt to the actual module names; GT-load and import failures fail fast with a clear message. The bottom of the table prints an explicit **REGRESSION CHECK** comparing `gpu`/`cpu`/`baseline` on `n_pred`, `f1@0.25`, `f1@0.5`, `tol_f1` (exact match within 1e-6) ? PASS iff all three agree, which is the "gpu ? cpu ? baseline, no regression" answer you're after. Nothing was run.